
title: "Nano-Kernel Runtime (NKR): A Bare-Metal Micro-VM Hypervisor for Multi-Tenant SaaS Workloads" subtitle: "White Paper — Version 1.6.9" date: "May 2026" lang: en geometry: "margin=2.5cm" fontsize: 11pt linestretch: 1.25 mainfont: "DejaVu Serif" sansfont: "DejaVu Sans" monofont: "DejaVu Sans Mono" colorlinks: true linkcolor: "NavyBlue" urlcolor: "NavyBlue" toc: true toc-depth: 2 numbersections: true header-includes:

- `\usepackage{microtype}`
- `\usepackage{fancyhdr}`
- `\pagestyle{fancy}`
- `\fancyhf{}`
- `\fancyhead[L]{\small Nano-Kernel Runtime (NKR) — White Paper}`
- `\fancyhead[R]{\small May 2026}`
- `\fancyfoot[C]{\thepage}`
- `\usepackage{booktabs}`
- `\usepackage{longtable}`
- `\renewcommand{\arraystretch}{1.3}`
- `\usepackage{listings}`
- `\lstset{basicstyle=\ttfamily\small, breaklines=true, frame=single, backgroundcolor=\color{gray!10}}`
- `\usepackage{xcolor}`

`\newpage`

Abstract. The *Nano-Kernel Runtime* (NKR) is a **proprietary** bare-metal hypervisor written in Rust that replaces container runtimes like Docker with hardware-isolated micro-VMs, running directly on Linux KVM. NKR is designed for operators managing dense multi-tenant SaaS deployments —especially Odoo ERP— on a single server with limited resources (16–32 GB RAM). By eliminating the overhead of QEMU, libvirt, and container-level sharing, NKR achieves full hardware isolation with a binary of just 2–4 MB, sub-second VM boot, exclusive CPU scheduling (the "chrs" model), and a Docker-compatible workflow for building disk images. Version 1.1 added six key capabilities: live filesystem sharing via VirtIO-FS, controlled CPU bursting via cgroupv2, L2 network isolation with ebttables, per-tenant database limits, a native Prometheus metrics exporter, and automatic multi-tenant compose file generation. Version 1.2 introduced four further optimizations targeting 100+ Odoo instances on 32 GB RAM: VirtIO-PMEM + DAX (eliminating ~150–200 MB of duplicated page cache per instance), async I/O via `io_uring` (reducing syscall overhead ~~70% under high-concurrency~~), ~~uncompressed ELF vmlinux loading~~ (20 ms faster boot), and a Seccomp BPF jailer. Version 1.3 made a leap in performance, density and operability with: the **Cell System** (multi-VM stacks with per-cell L2/L3 isolation), VirtIO-FS with DAX (replacing VirtIO-9P, 3–5× faster file I/O), VirtIO-Balloon (idle RAM reclamation), a VirtIO-Console channel (hvc0) for coordinated shutdown in ~2s, and instance cloning

(`nkr cell clone`). Version 1.4 stabilizes multi-tenant operation: *VirtIO-PMEM* active by *default*, *skip warmup* on clones, *filestore rename* inside the guest (no host-side loop-mounts), serialization of netlink operations (`flock + iptables -w`), and validation *hardening* across all API edges. Version 1.5 introduces **privilege separation**: the `nkr` daemon runs as root over a UDS socket (`/var/run/nkr.sock`) and the entire HTTP frontend lives in `nkr-api-server`, an *unprivileged* process (user `nkr-api`, no capabilities) whose only job is to translate HTTP ↔ IPC. An RCE in the HTTP parser does not compromise KVM/cgroups/iptables. Version 1.6 closes the multi-tenant loop with a complete external-panel-driven HTTP API: cloning Odoo tenants from a canonical *template seed* (created via `/web/database/create`, with Latin American Spanish pre-loaded), *edition* opt-in per-instance to enable/disable the enterprise share, *admin user password* applied via JSON-RPC at tenant boot (closing the inherited `admin/admin` window), automatic explosion of multi-module OCA repos to `addons/<module>/`, *server-side static caching in nginx* (`/web/static/* 24h`, `/web/assets/<hash>/* 30d`) with a `POST /admin/cache/purge` endpoint for explicit invalidation, *rate limit* on `/web/login`, and 444 (TCP close) over CMS/legacy paths. All of this lives in a single binary with no accessory daemons and is configurable via four documented REST endpoints (`/instances`, `/dns`, `/addons/git`, `/config`). Version 1.6.1 introduces the **tier system** (`production / staging / dev`), an `HVC0 REL_OD` control channel that reloads Odoo workers in ~3s without restarting the VM (resolving the `virtio-fs + inotify` limitation), and an *edge dual* nginx mode (Cloudflare proxied + DNS-only transparently coexisting via `set_real_ip_from`). Version 1.6.2 closes the density doctrine on 32 GB hosts with three pieces: **dynamic IDLE/ACTIVE ballooning** with automatic decay (the VM boots in maximum-squeeze state and deflates in ~2s on the first `SIGUSR2` from the panel), **Double Hygiene** on `POST /addons/git` (`git clean -ffdx recursive + full wipe of addons/` before re-populating — the tenant becomes a deterministic mirror of the meta-repo) with **strict 422 validation** of submodules, and the **chattr +i cycle** on master rootfs (`nkr build` applies `-i → build → +i` automatically), eliminating an entire class of failures from accidental mutation of the *immutable master*. Version 1.6.3 adds a **watchdog** (TCP probe of `:8069` per running tenant; auto-restart after 60s unresponsive — currently shipped *disabled* by env var while the panel pushes changes actively), corrects the tier doctrine by **emptying dev_mode** in DEV/STAGING (`reload` exhausts the guest's `inotify` watches under `virtio-fs` → `ENOSPC` → `respawn` loop; `qweb.xml` triggers Odoo's internal template-recompile on every request → CPU spikes → hangs; the canonical reload path is `REL_OD` over `HVC0`, not `dev_mode`), and raises the DEV profile to 1300 MB (soft/hard 800/1000) after observing `Server memory limit reached` cycling with Odoo 19 + ~31 custom modules in threaded mode. Version 1.6.4 closes a security/operability sprint: **HMAC-signed SSO** (`/nkr-sso?u=...&exp=...&sig=...` — the panel mints a 30s-TTL URL signed with a per-tenant 256-bit key in `odoo.conf [nkr_sso] secret`; an Odoo module verifies it constant-time and creates a sudo session without ever exposing the user's password to the host), **systemouts-addons** (a cell-level read-only addons directory mounted ahead of the tenant's `addons/` in `addons_path` — internal modules like `nkr_sso` live there, invisible to the client and untouched by `POST /addons/git`, distributed once per cell and inherited by clones via the template DB), **async POST /instances** (validates synchronously, dispatches the clone in the background, returns 202 + a `create-status` polling endpoint — eliminates the false 504 on slow PROD prefork boots), an **initramfs /mnt tmpfs** (any new `virtio-fs` share under `/mnt/*` "just works")

without rebuilding the master rootfs), the **POST /reload fix for workers=0** (the HVC0 watcher detects threaded vs prefork from `odoo.conf` → `kill -TERM` + supervisor respawn vs `kill -HUP` master), the **dynamic-balloon fix for tier=dev** (the balloon MMIO device is advertised to the guest even when `balloon_mb=0` at boot, so IDLE-decay inflation actually happens; `nr_balloon_mb` now reflects the runtime target), a **per-instance boot console log**, and a **kernel cmdline truncation fix** (omit redundant rootfs `nr.fs0/fsm0/fsr0` params when many virtio-fs shares are declared). Versions 1.6.5–1.6.9 harden production operation on the actual provisioned hardware (a 64 GB host): a **deterministic code reload** (the guest supervisor writes Odoo's PID to `/tmp/odoo.pid` and the HVC0 watcher issues a direct `kill -KILL` — a consistent 7s commit→reload cycle under real load), the ~~compose up → d detach fix~~ (VMs `setsid()` and write straight to the boot log → the compose process exits cleanly and VMs are reparented to init, no cascade kill from the previous pipe bug), the ~~enabled, REL_OD-aware watchdog~~ (120s threshold, 240s grace during deploys, with pre-restart forensics dumped to `/mnt/nkr/hangs/`), a ~~per-tier PostgreSQL connection policy~~ (dev/staging to direct PG with a bounded `db_maxconn`, prod via PgBouncer — after diagnosing that the chronic dev/staging hang was Odoo connection-pool exhaustion, not a kernel or memory problem), a ~~72% initramfs reduction~~ (1.17 MB/VM) by baking a monolithic, module-free kernel, the ~~symlinking of the pg/pgbouncer rootfs to the immutable master~~ (400 MB RSS + ~914 MB disk saved), and the **measurement of a real PSS baseline** that led to **evaluating and dropping** the snapshot/restore refactor as not worth its cost. This document presents the full architecture, implementation, and production deployment model of NKR v1.6.9.

\newpage

Introduction and Motivation

The Problem

Service providers managing dozens of SaaS tenants on shared infrastructure face a fundamental tension between **density** (maximizing the number of tenants per server) and **isolation** (avoiding the noisy neighbor effect). Docker containers offer high density but share the host kernel, exposing a large attack surface and lacking strict CPU or RAM guarantees. Traditional VMs (QEMU/KVM with libvirt) provide solid isolation but impose prohibitive memory and disk overhead for dense deployments.

Consider a practical scenario: an operator managing **50 Odoo 17 ERP instances** on a single 16–32 GB server using Docker:

Issue	Impact with Docker / Odoo.sh	Impact with NKR
Isolation	Shared (Container / PaaS)	Hardware-level (KVM Micro-VM)
CPU guarantee	Shared pools (no strict pinning)	Deterministic (pinned chrs)
RAM usage	Redundant (duplicated page cache)	Optimized (VirtIO-PMEM + DAX)

Issue	Impact with Docker / Odoo.sh	Impact with NKR
Restart latency	~3 minutes per stack	~2 seconds (via hv0)
Infra footprint	N Odoo + N PG + N nginx	N Odoo + 1 PG + 1 PgBouncer + 1 nginx

NKR was created to remove these tradeoffs, providing VM-grade isolation with the operational simplicity of a container.

What NKR Is

Nano-Kernel Runtime (NKR) is a purpose-built hypervisor that:

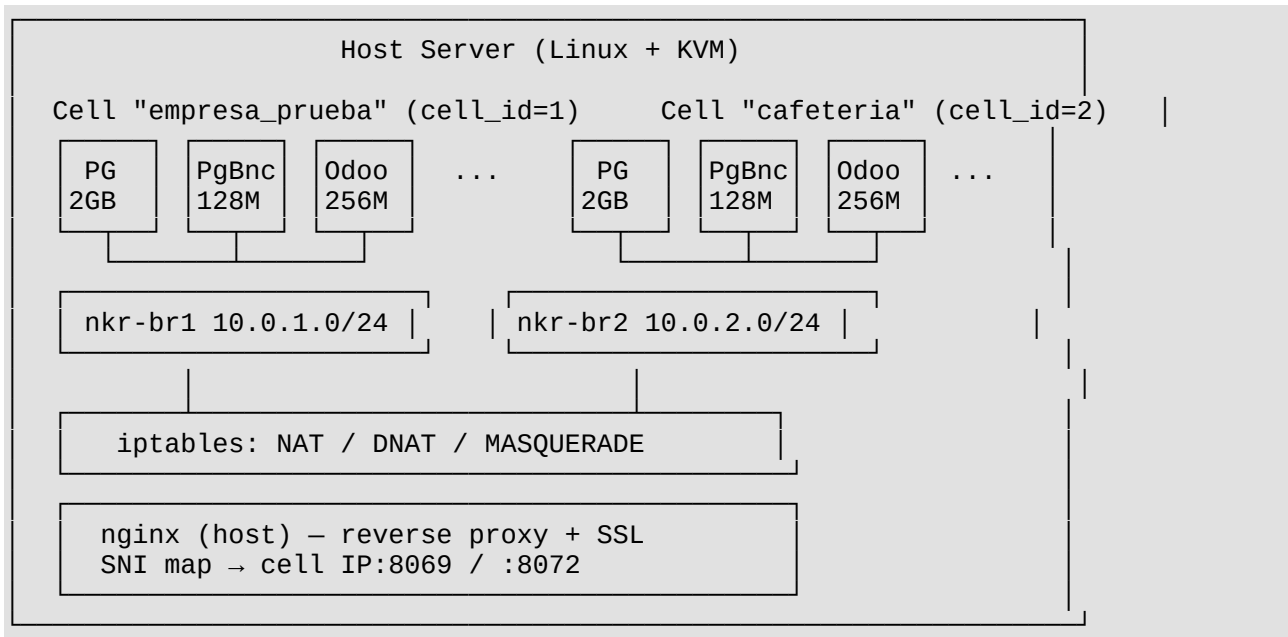
- Runs micro-VMs directly on `/dev/kvm` without QEMU, libvirt, or containerd
- Gives every "container" a real Linux kernel, an ext4 filesystem, and VirtIO devices
- Compiles to a **single ~2–4 MB binary** (Rust, LTO, *stripped*)
- Provides a Docker-compatible CLI (`nr run`, `nr ps`, `nr stop`, `nr restart`, `nr compose up`)
- Manages **Cells**: multi-VM groups with isolated L2/L3 networks (`nr cell create/up/down/clone`)
- Uses Docker **only** at build time to generate disk images from OCI/Dockerfiles

Design Goals

NKR's design is guided by five principles:

1. **Zero external runtime dependencies.** The `nr` binary requires only a Linux kernel with KVM support. No QEMU, no libvirt, no *container runtime*.
2. **Hardware isolation with container ergonomics.** Every workload runs in a complete KVM virtual machine—with its own kernel, page tables, and interrupt controller—while operators interact with it via familiar Docker-style commands and compose files.
3. **Deterministic resource allocation.** RAM is mapped exclusively to each VM. CPU cycles are guaranteed via *core pinning*. There is no *overcommit*.
4. **Minimal footprint.** The hypervisor binary is 2–4 MB. Guest overhead is bounded: a 256 MB VM uses exactly 256 MB of host RAM.
5. **Production-ready for multi-tenant SaaS.** First-class support for multi-tenant Odoo deployments with shared PostgreSQL (backed by PgBouncer), hot module updates, automated provisioning, and per-Cell network isolation to run multiple Odoo versions in parallel.

Architecture Overview



Every micro-VM is a complete virtual machine with:

- A Linux kernel (highly optimized `nanolinux` ELF or classic `bzImage`, binary shared across all VMs in a cell)
- An ext4 root filesystem (built from OCI images), optionally exposed via VirtIO-PMEM + DAX
- VirtIO-MMIO devices for block storage, network, VirtIO-FS with DAX, persistent memory, balloon, and console
- An initramfs that handles module loading, network configuration, VirtIO-FS mounting, and rootfs pivot
- Exclusive RAM and CPU *pinning* via `cgroupv2` + `sched_setaffinity`

VMM Engine: From KVM to Boot

The VMM engine (`vmm.rs`, ~2,600 lines) implements the full lifecycle of a micro-VM using direct KVM ioctls through the `rust-vmm` crate ecosystem — the same foundation used by AWS Firecracker and Intel Cloud Hypervisor.

KVM Initialization

1. Open `/dev/kvm`
2. `KVM_CREATE_VM` → VM file descriptor
3. `KVM_CREATE_IRQCHIP` → in-kernel PIC + IOAPIC
4. `KVM_CREATE_PIT2` → Programmable Interval Timer
5. Map guest memory → `GuestMemoryMmap` (two RAM regions; optional PMEM slot)
6. `KVM_CREATE_VCPU` → single vCPU (`id=0`)
7. Configure `CPUID`, `SREGs`, General Registers

Guest Memory Map (x86_64)

NKR uses a two-region memory model compatible with the Linux boot protocol:

Address	Contents	Size
0x0000–0x9FFFF	Base (conventional) RAM	640 KB
0x0500	GDT (<i>Global Descriptor Table</i>)	32 bytes
0x7000	<i>Zero Page</i> (boot parameters)	4 KB
0x9000	PML4 (<i>Page Map Level 4</i>)	4 KB
0xA000	PDPTE (<i>Page Directory Pointer</i>)	4 KB
0xB000	PDE (<i>Page Directory</i> , 2 MB pages)	4 KB
0x20000	Kernel command line	variable
0x100000	bzImage load address	~10 MB
0x800_0000	Initramfs	variable
0x1_0000_0000	VirtIO-PMEM slot (if --pmem)	= disk size
0x2_0000_0000	VirtIO-FS DAX window (if --share)	4 GB

Boot Protocol

NKR supports kernel formats auto-detected by file magic:

- **ELF nanolinux** (default): Detected by \x7fELF magic. Loaded via `linux-loader::Elf::load()`. The vCPU starts directly in 64-bit long mode (EFER=0xD01, CR0=0x80050033, CR4=PAE, CS.l=1). Eliminates gzip decompression in the guest entirely, dramatically speeding up boot.
- **bzImage** (classic v1.0): 32-bit Linux boot protocol. Kernel loaded at 0x100000 via `linux-loader::BzImage::load()`. The vCPU starts in 32-bit protected mode.

Boot sequence (shared):

1. **Kernel load** — ELF (in chunks) or bzImage loaded at 0x100000
2. **Initramfs load** — Copied to 0x800_0000 in guest memory
3. **Zero page setup** — Boot parameters at 0x7000
4. **Page table writes** — 2 MB pages with identity mapping via PML4 → PDPT → PD
5. **GDT writes** — 4-entry table: null, code64, data, null
6. **vCPU config** — RIP = kernel entry point; sregs set up for 64-bit (ELF) or 32-bit (bzImage)

The command line configures all VirtIO devices inline:

```

console=ttyS0 panic=1 pci=off noapic nolapic clocksource=jiffies tsc=nowatchdog
virtio_mmio.device=4K@0xd0000000:5      # network
virtio_mmio.device=4K@0xd0001000:6      # disk 0
virtio_mmio.device=4K@0xd0002000:7      # disk 1 (if any)
virtio_mmio.device=4K@0xd0010000:8      # VirtIO-FS share 0 (if --share)
virtio_mmio.device=4K@0xd0020000:16     # PMEM (if --pmem)
virtio_mmio.device=4K@0xd0030000:17     # Balloon
virtio_mmio.device=4K@0xd0040000:18     # VirtIO-Console (hvc0)
root=/dev/vda rw init=/sbin/init nkr.ip=10.0.{cell_id}.{vm_id+1}
# With --pmem: root=/dev/pmem0 rootflags=dax

```

Time and Clock Management

Micro-VMs can suffer clock drift in dense environments. NKR addresses this with two mechanisms:

1. **PIT2 (Programmable Interval Timer):** Explicitly instantiated (KVM_CREATE_PIT2), providing the base interrupt time source vital for the *guest* scheduler.
2. **Kernel Clock Sources:** The `clocksource=jiffies tsc=nowatchdog` parameter forces the guest kernel to base time on jiffies (timer interrupts) and disables the TSC watchdog, preventing hangs from TSC inconsistencies under high CPU contention.

vCPU Loop

The main loop runs KVM_RUN continuously, handling four exit types:

Exit type	Handler
IoOut (port I/O write)	Serial console output (COM1 0x3F8)
IoIn (port I/O read)	Serial status registers
MmioWrite	Writes to VirtIO-MMIO registers (device config, queues, notifications)
MmioRead	Reads from VirtIO-MMIO registers (features, status, config space)
Hlt / Shutdown	Clean exit

Robust Shutdown and Restart — *New in v1.3*

VirtIO-Console (hvc0) control channel (`console.rs`): The guest init process blocks on `read -r cmd < /dev/hvc0`. On receiving "SHUTDOWN\n", init runs an orderly shutdown (SIGTERM to services, waits up to 25s for the PostgreSQL postmaster, calls `poweroff`).

SIGTERM flow in the VMM (`vmm.rs`:1916–1944):

1. SIGTERM received → SHUTDOWN_REQUESTED AtomicBool set
2. First loop iteration (phase=0):
 - Stores SHUTDOWN_STARTED_MS timestamp
 - Injects "SHUTDOWN\n" into the hvc0 receiveq

- Arms SIGALRM every 1s (`setitimer`) to break out of `vcpu.run()` if the guest is in HLT
 - Advances to `phase=1`
3. Subsequent iterations (`phase=1`):
 - Re-injects "SHUTDOWN\n" every 2s — mitigates races where the virtio-console driver init delayed reading the first inject
 - After 60s timeout: forced break of vCPU loop
 4. After break: extract RW volumes, tear down TAP, remove iptables rules, deregister state

Zombie detection (`state.rs:249–256`): `is_pid_alive()` combines `kill(pid, 0)` with reading `/proc/{pid}/status` to treat zombie processes (`State: Z`) as dead. This avoids `nkr stop/nkr restart` waiting 90s in vain when the parent compose process has not called `wait()`.

nkr restart (`main.rs:126–208`):

1. Reads `/proc/{pid}/cmdline` — captures the original `nkr run ... argv`
2. Stops the VM via SIGTERM (90s timeout, degrading to SIGKILL)
3. Waits 500ms — allows TAP/bridge cleanup to complete
4. Relaunches with `setsid()` (detached from terminal), stdout/stderr redirected to `/tmp/nkr-restart-{vm_id}.log`

Result: typical restart cycle in ~2s with the hvc0 channel, vs. 60s timeout if the channel is unavailable.

Hot Code Reload via HVC0 (REL_OD) — *refined in v1.6.9*

The HVC0 channel is also used to **reload Odoo's code without restarting the whole VM**. An external trigger (in production, the panel) signals the deploy and the VMM injects `REL_OD\n` into the hvc0 receiveq; a watcher in the guest (running *outside* the rootfs chroot) receives it and acts based on Odoo's mode, which it detects by reading `workers = N` from the guest's `odoo.conf`:

- **workers=0 (threaded)**: the guest supervisor launches Odoo with `su -c 'echo $$ > /tmp/odoo.pid; exec /usr/bin/python3 -u /usr/bin/odoo ...'` — the inner shell writes to `/tmp/odoo.pid` the PID that becomes Python's after the `exec`. The watcher reads `/newroot/tmp/odoo.pid` and issues a **direct kill -KILL \$pid**, with no `pkill -f` and no SIGTERM grace. SIGKILL is safe in threaded mode (there is no prefork master to preserve) and the supervisor's `while true` respawns with fresh code.
- **workers>0 (prefork)**: `pkill -HUP` to the master, which recycles the workers. For **new-code** deploys in PROD, recycling workers is not enough (the child processes' copy-on-write does not re-import the modules), so the PROD flow uses a VM restart (~15–19s).

Measured under real load (live websocket + running cron): a **consistent ~7s commit → reload cycle**

in threaded mode. Every watcher step is logged to `<instance>/logs/nkr-watcher.log` (visible from the host via the virtio-fs share) for live diagnosis.

The **compose up -d detach fix** (v1.6.9) is complementary: spawned VMs `setsid()` and redirect stdout/stderr **straight to the boot log file** (`Stdio::from(File)`, no pipes), so the `compose` process exits cleanly after the health checks and all VMs end up with `PPid=1` (reparented to `init`). The previous bug left `compose up -d` hung forever in `child.wait()` holding pipes — a `pkill -f 'nkr compose'` then killed **all** VMs in cascade.

VirtIO Device Model

NKR implements the VirtIO-MMIO (*Memory-Mapped I/O*) transport, not PCI, for maximum simplicity. The kernel boot parameter `pci=off` disables PCI enumeration entirely.

MMIO Address Map

Address	Device	IRQ	Since
0xD000_0000	VirtIO-Net (network)	5	v1.0
0xD000_1000	VirtIO-Block disk 0 (rootfs)	6	v1.0
0xD000_2000	VirtIO-Block disk 1	7	v1.0
0xD000_3000+	Additional disks (+0x1000 each)	8+	v1.0
0xD001_0000+	VirtIO-FS shares (+0x1000 each, DAX)	8, 9, 10, ...	v1.3
0xD002_0000	VirtIO-PMEM (persistent memory, DAX)	7	v1.2
0xD004_0000	VirtIO-Balloon	10	v1.3
0xD005_0000	VirtIO-Console (hvc0)	11	v1.3

The `0xD001_0000+` range guarantees no collision with the block zone (which grows up to `0xD000_9000` with 9 disks). PMEM, Balloon, and Console are statically reserved. The guest is PIC-only (16 legacy IRQs, no APIC), so IRQs above the first few are *shared* across virtio-mmio devices — virtio-mmio's per-device interrupt-status register disambiguates which device fired, so sharing is safe (verified: all devices register `DRIVER_OK` regardless of IRQ sharing).

VirtIO Block Device

Implementation: `block.rs` — ~310 lines

- **Queue:** Single virtqueue, 256 descriptors
- **Sector size:** 512 bytes

- **Operations:** Read (type=0), Write (type=1)
- **Descriptor chain:** Header (16 bytes: type + sector) → Data buffer → Status byte
- **Interrupts:** IRQ injection via `irqfd` after processing each completion batch
- **Feature negotiation:** `VIRTIO_F_VERSION_1` (bit 32)
- **Async I/O (v1.2):** Uses `io_uring` (depth 128) for non-blocking reads/writes. Each operation is submitted as an `opcode::Read` or `opcode::Write` SQE; completions are drained at the start of each vCPU loop iteration via `poll_completions()`. Silent fallback to synchronous `pread64/pwrite64` if `io_uring` is unavailable (kernel < 5.1).

VirtIO Network Device

Implementation: `net.rs` — ~310 lines

- **Queues:** Two virtqueues (RX=0, TX=1), 256 descriptors each
- **Backend:** Linux TAP device (`/dev/net/tun`, `TUNSETIFF`)
- **Header:** 12-byte VirtIO-net header (stripped/added on TX/RX)
- **RX path:** A dedicated background thread does blocking reads from the TAP fd, injects packets into the RX queue, and signals the IRQ
- **TX path (v1.2):** `opcode::Write` SQE to the TAP fd via `io_uring` (depth 64). The `Vec<u8>` payload is held in `tx_pending` until the CQE is drained, preventing premature deallocation. Falls back to `tap.write_all()` if the ring is unavailable.
- **Features:** `VIRTIO_NET_F_MAC` | `VIRTIO_NET_F_STATUS` | `VIRTIO_F_VERSION_1`

VirtIO-FS Device (*File Sharing*) — New in v1.3

Implementation: `virtio_fs.rs`

NKR v1.3 replaces the VirtIO-9P server with VirtIO-FS with DAX, delivering 3–5× faster filesystem access for Python library loading and Odoo hot module updates.

- **Protocol:** vhost-user socket connecting to the external `virtiofsd` daemon
- **DAX window:** 4 GB mounted at guest physical address `0x2_0000_0000` (KVM slot 3). Guest reads access the host *page cache* directly with no extra copy.
- **Device ID:** 26 over VirtIO-MMIO
- **Semantics:** Full POSIX compatibility (`fcntl`, `O_DIRECT`, `flock`)
- **CLI:** `--share host_path:guest_path` (repeatable; first share at `0xD001_0000`, each additional `+0x1000`)
- **Performance:** Cold boot of 30 micro-VMs sharing a common Odoo rootfs drops from ~90s (9P) to ~25s (VirtIO-FS DAX)

In the guest, the `initramfs` automatically mounts shares declared on the kernel `cmdline`:

```
nkr run --disk odoo.ext4 --share /opt/modules:/mnt/extra-addons
# guest: mount -t virtiofs virtiofs0 /mnt/extra-addons
```

VirtIO-PMEM + DAX Device — New in v1.2

Implementation: `pmem.rs` — ~200 lines

VirtIO-PMEM (device ID 27) maps the guest root disk into host RAM via `mmap(MAP_SHARED)` and registers it as a KVM memory slot at guest physical address `0x1_0000_0000` (4 GB). The guest kernel (with `CONFIG_VIRTIO_PMEM=y` and `CONFIG_FS_DAX=y`) exposes it as `/dev/pmem0` and mounts the rootfs with the `dax` option, fully eliminating the guest page cache.

- **MMIO:** `0xD002_0000`, IRQ 16, device ID 27
- **Config space (offset 0x100):** `[u64 start_phys_addr][u64 size]`
- **KVM slot:** Slot 2 (slots 0 and 1 are used by the two RAM regions)
- **Host mmap:** `MAP_SHARED | PROT_READ | PROT_WRITE + MADV_HUGEPAGE` hint to reduce TLB pressure
- **FLUSH requests:** Guest sends `VIRTIO_PMEM_REQ_TYPE_FLUSH`; NKR responds with `msync(MS_ASYNC)` without blocking the vCPU
- **E820 entry:** Type 12 (*persistent memory*) for `[4 GB, 4 GB + disk_size]`
- **Cmdline change:** `root=/dev/pmem0 rootflags=dax` replaces `root=/dev/vda rw`
- **Fallback:** Silent to VirtIO-Block if the disk cannot be mmap'd
- **Guest kernel requirement:** `CONFIG_VIRTIO_PMEM=y`, `CONFIG_FS_DAX=y`
- **CLI:** `--pmem` flag on `nkr run`

Memory savings mechanism: With DAX, guest reads access the host page cache directly — no second copy of the data exists in guest RAM. For a typical Odoo instance with an active *working set* of ~300 MB read, this eliminates 150–200 MB of duplicated page cache per VM.

VirtIO-Balloon Device — New in v1.3

Implementation: `balloon.rs` — ~150 lines

VirtIO-Balloon (device ID 5) reclaims unused memory from idle VMs and returns it to the host kernel via `madvise(MADV_DONTNEED)`.

- **MMIO:** `0xD003_0000`, IRQ 17
- **Operation:** The VMM writes the balloon target (in pages) to the device config space; the guest driver inflates/deflates by allocating/freeing pages
- **CLI:** `--balloon-mb N` on `nkr run` pre-inflates the balloon by N MB at boot

- **Combined effect:** A 700 MB VM with `-balloon-mb 300` effectively occupies only ~400 MB of host RAM
- **Compose:** `balloon_mb: 200` in the service spec

Density comes from coordinating VirtIO-FS+DAX (dedupes binaries, libs and `.pyc` files across VMs by reading from the same host backing file), VirtIO-PMEM+DAX (eliminates the duplicated page cache of the RO rootfs in every guest), and VirtIO-Balloon (reclaims idle RAM and returns it to the host via `MADV_DONTNEED`). **KSM does not contribute to these savings in v1.4+:** the VMM maps guest memory using `memfd_create + MAP_SHARED` (a hard requirement of the `vhost-user SET_MEM_TABLE` protocol used by `virtiofsd`), and the kernel silently rejects `madvise(MADV_MERGEABLE)` on VMAs flagged `VM_SHARED`, returning `EINVAL`. As a result, `nkr stats` reports `[KSM] state=stopped | shared=0 | savings≈0MB`. The real density gain comes from DAX, not from KSM.

VirtIO-Console (hvc0) Device — New in v1.3

Implementation: `console.rs`

VirtIO-Console provides a bidirectional control channel between the VMM and the guest init process, used exclusively for coordinated shutdown.

- **Device ID:** 3 over VirtIO-MMIO at `0xD004_0000`, IRQ 18
- **Guest side:** Init blocks on `read -r cmd < /dev/hvc0`. On `"SHUTDOWN\n"`: orderly shutdown (SIGTERM to services, wait for PG postmaster, `poweroff`)
- **Host side:** `try_inject(b"SHUTDOWN\n")` writes to the `receiveq` and raises the IRQ. `poll_pending()` retries if the queue was full
- **Race mitigation:** The VMM re-injects every 2s during the shutdown window in case the first injection was lost before the `hvc0` driver was initialized

Networking

Bridge Topology

NKR supports two network modes:

Legacy mode (`cell_id=0`): Single bridge `nkr0`, subnet `10.0.0.0/24`. All VMs share an L2 domain.

Cell mode (`cell_id=1..254`): Per-cell bridge `nkr-br{N}`, subnet `10.0.{N}.0/24`. Each cell is an isolated L2/L3 domain with its own NAT.

Legacy (<code>cell_id=0</code>): (cafeteria):	Cell 1 (<code>empresa_prueba</code>):	Cell 2
<code>nkr0 10.0.0.0/24</code>	<code>nkr-br1 10.0.1.0/24</code>	<code>nkr-br2 10.0.2.0/24</code>
<code>nkr-tap1 → VM 10.0.0.2</code>	<code>nkr-c1-tap1 10.0.1.2</code>	<code>nkr-c2-tap1 10.0.2.2</code>
<code>nkr-tap2 → VM 10.0.0.3</code>	<code>nkr-c1-tap2 10.0.1.3</code>	<code>nkr-c2-tap2 10.0.2.3</code>

Deterministic Formula

Defined in `src/registry.rs:216`:

```
IP  = 10.0.{cell_id}.{vm_id + 1}
MAC = 52:54:00:{cell_id}:34:{vm_id}
TAP = nkr-c{cell_id}-tap{vm_id}    (cell_id>0)
      = nkr-tap{vm_id}              (cell_id=0, legacy)
```

Conventional per-cell assignments: `pg=vm_id 1`, `pgbouncer=vm_id 2`, `odoo-NN=vm_id 3..N`.

Example `cell_id=1`: `db → 10.0.1.2`, `pgbouncer → 10.0.1.3`, `odoo-01 → 10.0.1.4`.

Automatic Setup

For every VM, the VMM (`vmm.rs:956–974`):

1. Creates the TAP device `nkr-c{cell_id}-tap{vm_id}` (or `nkr-tap{vm_id}` in legacy)
2. Attaches it to bridge `nkr-br{cell_id}` (or `nkr0`)
3. Assigns MAC `52:54:00:{cell_id}:34:{vm_id}`
4. Passes the IP to the guest via kernel cmdline (`nkr.ip=`)
5. Configures iptables rules:
 - `POSTROUTING MASQUERADE` for internet access
 - `FORWARD ACCEPT` for inter-VM traffic within the cell
 - `PREROUTING DNAT + OUTPUT DNAT` for port forwarding (if `--port`)

Rules are checked with `iptables -C` before adding (idempotent) and removed when the VM shuts down.

Port Forwarding

```
nkr run --disk odoo.ext4 --port 8069:8069 --id 2
# Creates: host:8069 → 10.0.0.3:8069 (DNAT + MASQUERADE)
```

L2 Isolation with ebtables — New in v1.1

Three ebtables rules per TAP confine traffic to the MAC+IP assigned by the hypervisor:

```
ebtables -A INPUT -i nkr-c1-tapN -p ARP --arp-mac-src 52:54:00:01:34:N -j ACCEPT
ebtables -A INPUT -i nkr-c1-tapN -p IPv4 --ip-src 10.0.1.(N+1) -s
52:54:00:01:34:N -j ACCEPT
ebtables -A INPUT -i nkr-c1-tapN -j DROP
```

These rules prevent a compromised VM from sending packets with a different MAC/IP. Rules are removed in cleanup via `teardown_tap_isolation()`. If `ebtables` is not installed, NKR emits a warning and continues without L2 isolation (silent fallback).

Cell System — New in v1.3

The Cell System is NKR's answer for running **multiple independent Odoo stacks** (e.g. Odoo 15, 17, and 19 for different customer types) on the same host without IP/network conflicts.

What Is a Cell?

A *cell* is a named group of micro-VMs with:

- A dedicated Linux bridge and subnet ($10.0.\{cell_id\}.0/24$)
- Its own `nkr-compose.yml` orchestrating the full stack (PG + PgBouncer + N Odoos)
- Isolated instance directories under `/mnt/nkr/cells/<name>/instances/`
- Cell-scoped VM registry (IDs 2–254 per cell, independent across cells)

Up to 254 cells can coexist on a single host (`cell_ids` 1–254). `cell_id=0` is the flat legacy mode.

Directory Structure

```
/mnt/nkr/
├── cell-registry.json          # cell_name → cell_id
├── registry.json             # "cell_name/vm_name" → vm_id (scoped)
└── cells/
    ├── empresa_prueba/      # cell "empresa_prueba" (cell_id=1)
    │   ├── cell.yml         # { name, cell_id, odoo_version }
    │   ├── nkr-compose.yml  # full-stack compose
    │   └── instances/
    │       ├── empresa_prueba-odoo-01/
    │       │   ├── config/odoo.conf
    │       │   ├── addons/
    │       │   ├── filestore/
    │       │   └── logs/
    │       ├── empresa_prueba-odoo-02/
    │       └── ...
```

Registry System

cell-registry.json — maps `cell_name` → `cell_id` (integer, 1–254). Persisted at `/mnt/nkr/cell-registry.json`.

registry.json — maps `"cell_name/vm_name"` → `vm_id` (integer, 2–254, scoped per cell). Persisted at `/mnt/nkr/registry.json`.

The scoped key format means `empresa_prueba/odoo-01` and `cafeteria/odoo-01` can both have `vm_id=3` without conflict — they live in different subnets ($10.0.1.4$ vs $10.0.2.4$).

`resolve_id_scoped(cell_name, vm_name)` in `registry.rs:106` allocates the next free ID within the cell scope, or returns the existing one if already registered.

`register_explicit_scoped()` records a specific ID and verifies it isn't already taken within the same cell scope.

Cell Lifecycle

```
# Create a cell – registers cell_id, creates bridge + directory structure
sudo nkr cell create empresa_prueba --odoo-version 17.0

# Generate compose (external script or manual) and start the full stack:
sudo nkr cell up empresa_prueba -d          # compose up in daemon mode

# Status
sudo nkr cell ls                            # table of all cells
sudo nkr cell ps empresa_prueba             # active VMs in this cell

# Shutdown
sudo nkr cell down empresa_prueba           # stops all VMs
sudo nkr cell destroy empresa_prueba        # removes from registry (data
preserved)
```

Cell Compose Format

Cell compose files include `cell_id` and `nkr_name` per service:

```
services:
  pg:
    nkr_name: "empresa_prueba-pg"
    id: 1
    disks: [/mnt/nkr/images/postgres.ext4]
    ram: 2048
    chrs: 5
    cell_id: 1
    ports: ["5432:5432"]

  pgbouncer:
    nkr_name: "empresa_prueba-pgbouncer"
    id: 2
    ram: 128
    chrs: 1
    cell_id: 1

  odoo-01:
    nkr_name: "empresa_prueba-odoo-01"
    id: 3
    ram: 512
    chrs: 2
    cell_id: 1
    environment:
      DB_HOST: "10.0.1.2"          # PG IP: 10.0.{cell_id}.{pg_vm_id+1}
      PGB_HOST: "10.0.1.3"        # PgBouncer IP
      DB_NAME: "db-empresa_prueba-odoo-01"
```

IPs in `environment`: are **literals** computed from `cell_id` + `vm_id` at compose generation time.

Instance Cloning — `nkr cell clone`

`clone_instance()` in `cell.rs:659` provides atomic cloning of an Odoo instance within a cell — the main path to create test/staging environments from production.

Algorithm:

1. Scans `cells/*/instances/<src>/` to locate the owning cell
2. Rejects if `dst` already exists
3. Warns if VM `src` is active (PG sessions will be briefly interrupted)
4. Registers `dst_vm_id` via `resolve_id_scoped` (next free ID in cell scope)
5. `cp -a --reflink=auto <src_dir> <dst_dir>` — O(1) on btrfs/XFS (CoW), physical copy on ext4
6. Cleans destination logs
7. `rewrite_odoo_conf()` — replaces every `src_nkr` → `dst_nkr` occurrence in `odoo.conf` (`db_name`, `dbfilter`, `paths`)
8. `clone_database()` — atomic PostgreSQL clone:
 - `ALTER DATABASE "{src}" WITH ALLOW_CONNECTIONS false`
 - `SELECT pg_terminate_backend(...)` — disconnects active sessions
 - `CREATE DATABASE "{dst}" WITH TEMPLATE "{src}" OWNER odoo`
 - `ALTER DATABASE "{src}" WITH ALLOW_CONNECTIONS true`
 - Connectivity verified with `pg_isready` before attempting; rollback on failure
9. `append_compose_block()` — YAML text edit (preserves comments and original formatting):
 - Locates the `src` service block by exact `nkr_name`:
 - Clones the block with new header, new `id:`, all `src_nkr` → `dst_nkr` substitutions
 - Creates a timestamped backup (`nkr-compose.yml.bak.{unix_ts}`)

Flags:

- `--no-db` — skips database cloning
- `--no-compose` — skips compose modification

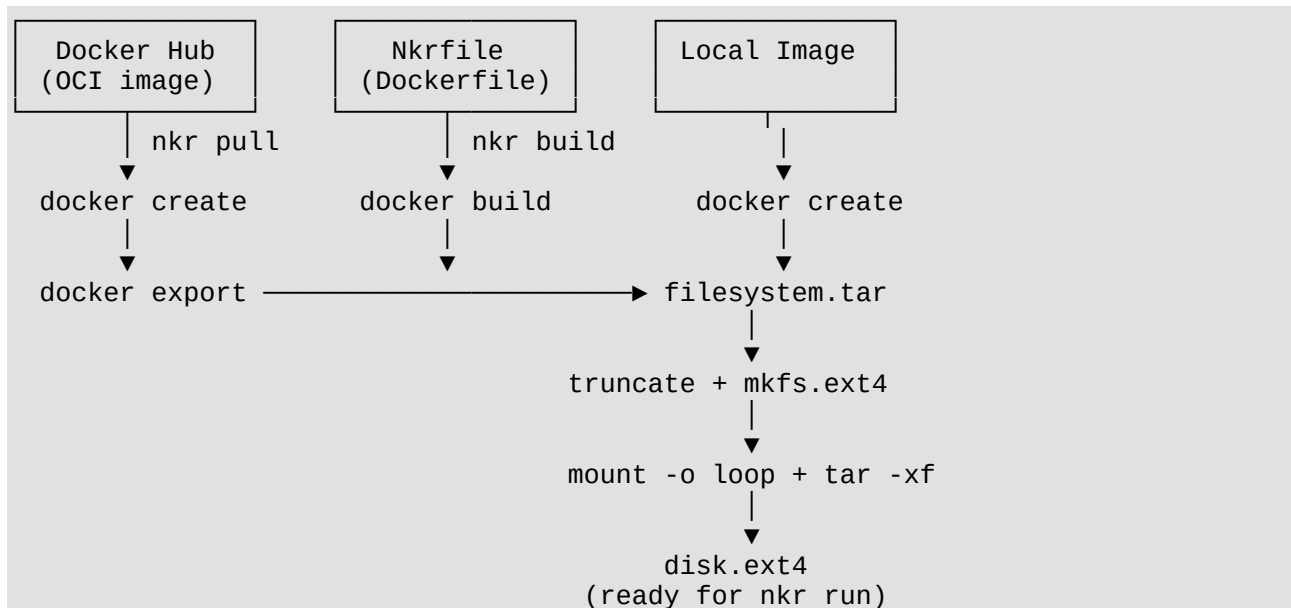
```
# Full clone (files + DB + compose)
sudo nkr cell clone empresa_prueba-odoo-01 empresa_prueba-odoo-04

# Safe smoke test (files only, no DB or compose)
sudo nkr cell clone empresa_prueba-odoo-01 empresa_prueba-odoo-04 --no-db --no-compose
```

Disk Lifecycle: From OCI to ext4

NKR uses Docker exclusively as a **build tool** to transform OCI images into raw ext4 filesystems. Docker is not required at runtime.

Build Pipeline



Nkrfile Format

Nkrfiles are standard Dockerfiles. NKR provides templates for common services:

```
# Nkrfile.pg -- PostgreSQL 15
FROM postgres:15
ENV POSTGRES_USER=odoo
ENV POSTGRES_PASSWORD=odoo

# Nkrfile.odoo -- Odoo 17
FROM odoo:17.0
USER root
COPY deploy/config/odoo.conf /etc/odoo/odoo.conf
RUN mkdir -p /mnt/extra-addons && chown odoo:odoo /mnt/extra-addons
USER odoo
```

Copy-on-Write Snapshots

For multi-tenant deployments, NKR creates CoW snapshots from a base disk:

```
cp --reflink=auto odoo-base.ext4 client1.ext4
```

On filesystems supporting reflinks (btrfs, XFS with reflink), this operation is instant and consumes no extra space. On others, NKR falls back to `cp --sparse=always`.

Volumes

NKR provides a volume system to inject configuration and persist data:

- **Pre-boot injection:** The root disk is loop-mounted and files are copied from host to guest paths
- **Post-shutdown extraction:** Volumes marked `:rw` are copied back from guest to host
- **Format:** `host_path:guest_path` (read-only) or `host_path:guest_path:rw`

Environment Variables

Environment variables are written to `/etc/nkr-env` inside the root disk before boot:

```
nkr run --disk pg.ext4 --env POSTGRES_USER=odoo --env POSTGRES_PASSWORD=secret
```

The initramfs loads this file during boot, making variables available to the guest init process.

The CPU Model: "Chrs"

NKR introduces a CPU allocation unit called the **chr** (pronounced "core"):

Value	Meaning
1 chr	20% of a physical core
5 chrs	1 full physical core
10 chrs	2 physical cores

Implementation

CPU allocation is enforced via `sched_setaffinity()`:

```
let cores_needed = ((chrs as f32) / 5.0).ceil() as u32;  
let cores_to_use = cores_needed.min(num_cpus);  
// Pin the vCPU thread to cores [0..cores_to_use]  
sched_setaffinity(0, &cpuset);
```

Chrs are **exclusive** — the VM process is pinned to dedicated physical cores, avoiding contention with other VMs.

CPU Bursting with cgroupv2 — New in v1.1

NKR v1.1 adds controlled CPU bursting via the cgroupv2 `cpu.max` controller. The minimum guarantee remains `1 chr = 20% of a core`, but the VM can absorb idle host cycles without impacting other tenants.

```
cgroupv2 config for N chrs:  
  cpu.max      → "{N×20000} 100000"    (N×20% quota every 100 ms period)  
  cpu.max.burst → "{N×5000}"           (extra accumulable credit – kernel ≥  
5.15)
```

The hierarchy is created at `/sys/fs/cgroup/nkr/{vm-name}/` and removed on VM shutdown via `teardown_cgroup()`.

nkr nitro — Temporary CPU Unblock

```
nkr nitro empresa_prueba-odoo-01 --duration 10m
```

Writes `max 100000` to the VM's `cpu.max`, giving it unlimited CPU for the specified duration (default 10m). A detached `sh -c "sleep N; echo quota > cpu.max"` (detached with

`setsid()` restores the throttle. Useful when installing heavy Odoo modules (`-i account, mrp, website`).

Dynamic Nitro During Compose Boot

During `compose up`, every service with `healthcheck:` goes through an automatic CPU cycle:

1. **`nitro_relax_cgroup()`** — sets `cpu.max = max 100000` when starting the VM (full CPU during boot)
2. **TCP health check** — waits for the service port to accept connections
3. **`run_warmup()`** — issues HTTP GETs to `/web/assets/debug/*.{css,js}` and `/web/login` to force QWeb asset compilation before the first real client
4. **30s grace period** — keeps CPU at max for the first backend request
5. **`nitro_throttle_cgroup()`** — restores the configured `chrs` quota

Logs: `[NKR-WARMUP] □ X compiled (Ts, N bytes)` for every compiled asset.

Disk I/O Limiting with `cgroupv2` — New in v1.1

The same `cgroupv2` hierarchy enforces I/O rate limits per block device:

```
io.max → "MAJ:MIN rbps=209715200 wbps=104857600" (200 MB/s read, 100 MB/s write)
```

Device numbers (major:minor) are obtained via `libc::stat()` on the disk path. Enforcement is done by the kernel's `blk-mq` scheduler, with no additional CPU cost in the hypervisor.

Sample deployment (8-core server):

Service	Chrs	Cores Used
PostgreSQL	10 (2 cores)	Cores 0–1
Odoo #1	5 (1 core)	Core 2
Odoo #2	5 (1 core)	Core 3
Odoo #3–#8	1 each	Cores 4–7 (shared)

Initramfs Generation

NKR includes an automatic initramfs generator (`initramfs.rs`, ~920 lines) that creates boot environments tailored to each service.

Boot Sequence

```
The initramfs boots (PID 1)
├─ Mount /proc, /sys, /dev
```

```

- Load kernel modules:
  crc32c → libcrc32c → crc16 → mbcache → jbd2 → ext4
  virtio_blk → failover → net_failover → virtio_net
  fuse → virtiofs (if VirtIO-FS shares declared – v1.3)
  virtio_pmem → nd_btt → dax (if --pmem active – v1.2)

- Wait for /dev/vda or /dev/pmem0 (up to 3 seconds)
- Parse nkr.ip= from /proc/cmdline
- Configure eth0: IP/24, default route → 10.0.{cell_id}.1

- Mount /dev/vda (or /dev/pmem0 with dax) → /newroot (ext4)
- Mount additional disks /dev/vdb..vde → /newroot/mnt/disk0..3
- Mount VirtIO-FS units (if on cmdline – v1.3):
  mkdir -p /newroot${NKR_FS0_MNT}
  mount -t virtiofs virtiofs0 /newroot$mnt
- Bind-mount /proc, /sys, /dev into /newroot

- Write /etc/nkr-net.sh (network config script)
- Write /etc/resolv.conf (DNS: 8.8.8.8, 8.8.4.4)
- Configure network via chroot

- Detect init: /sbin/init → systemd → Docker entrypoint
- Create wrapper /sbin/nkr-init:
  - Load /etc/nkr-env (NKR environment variables)
  - Start hvc0 watcher: read -r cmd < /dev/hvc0 (blocking)
  - Exec the detected init

- exec switch_root /newroot /sbin/nkr-init

```

Automatic Entrypoint Detection

When built with `nkr pull` or `nkr build`, NKR:

1. Extracts `ENTRYPOINT` + `CMD` from Docker image metadata
2. Mounts the disk read-only and looks for known entrypoint scripts (`/entrypoint.sh`, `/docker-entrypoint.sh`, etc.)
3. Generates a custom init script that loads NKR environment variables and launches the correct entrypoint

This lets NKR boot unmodified Docker images — PostgreSQL, PgBouncer, nginx, Redis, Odoo — as micro-VMs with no image changes.

Compose Orchestration

NKR provides a compose system (`compose.rs`, ~1,600 lines) modeled on Docker Compose but designed for VM orchestration.

Compose File Format

```

services:
  db:
    nkr_name: "empresa_prueba-pg"
    id: 1
    cell_id: 1

```

```

disks: [/mnt/nkr/images/postgres.ext4]
ram: 2048
chrs: 5
ports: ["5432:5432"]
shares:
["/mnt/nkr/cells/empresa_prueba/pg/data:/var/lib/postgresql/data:rw"]
healthcheck:
  port: 5432
  initial_delay: 15
  interval: 5
  retries: 12

odoo-01:
  nkr_name: "empresa_prueba-odoo-01"
  id: 3
  cell_id: 1
  disks: [/mnt/nkr/images/odoo.ext4]
  ram: 512
  chrs: 2
  ports: ["8069:8069", "8072:8072"]
  shares:
  -
  - "/mnt/nkr/cells/empresa_prueba/instances/empresa_prueba-odoo-01/filestore:/var/lib/odoo:rw"
  -
  - "/mnt/nkr/cells/empresa_prueba/instances/empresa_prueba-odoo-01/addons:/mnt/extra-addons"
  environment:
    DB_HOST: "10.0.1.2"
    DB_NAME: "db-empresa_prueba-odoo-01"
  healthcheck:
    port: 8069
    initial_delay: 20
    interval: 5
    retries: 24

```

Sequential Boot Order

Compose starts services in dependency order:

1. db — PostgreSQL, waits for TCP probe at :5432
2. pgbouncer — waits for TCP probe at :6432
3. All odoo- * services — launched in parallel once PgBouncer is healthy

Resource Resolution

NKR compose resolves resources intelligently, following a priority chain:

Resource	Resolution Order
Disk	YAML path → <yaml_dir>/<name> → /mnt/nkr/images/<name>
Kernel	Explicit → <yaml_dir>/nanolinux → /mnt/nkr/kernel/nanolinux → next to nkr binary
Initramfs	Explicit → by service name → by disk name →

Resource	Resolution Order
	heuristic → auto-generated

Features

- **Auto-build:** If a service has a `build:` section and the disk doesn't exist, it's built automatically
- **Health checks:** TCP monitoring with configurable delay, interval, and retries
- **Daemon mode:** `nkr compose up -d` runs in the background with log rotation (max 10 MB, 3 rotated)
- **CoW snapshots:** Automatic snapshot creation when a base disk is already in use by another VM
- **Deterministic IDs:** Services use `nkr_name` + optional `id:`; cell-scoped IDs in `registry.json`
- **Warmup + dynamic Nitro:** Automatic CPU relaxation during boot, QWeb asset pre-compilation, 30s grace period

NKR Data Directory

```

/mnt/nkr/
├── images/
├── initramfs/
│   ├── base/
│   ├── pg.cpio.gz
│   └── odoo.cpio.gz
├── kernel/
├── snapshots/
├── cell-registry.json
├── registry.json
├── cells/
│   └── empresa_prueba/
│       ├── cell.yml
│       ├── nkr-compose.yml
│       └── instances/
└── # Default (NKR_DATA_DIR variable)
    # Base ext4 disk images
    # .cpio.gz files per service
    # busybox + kernel modules (shared)

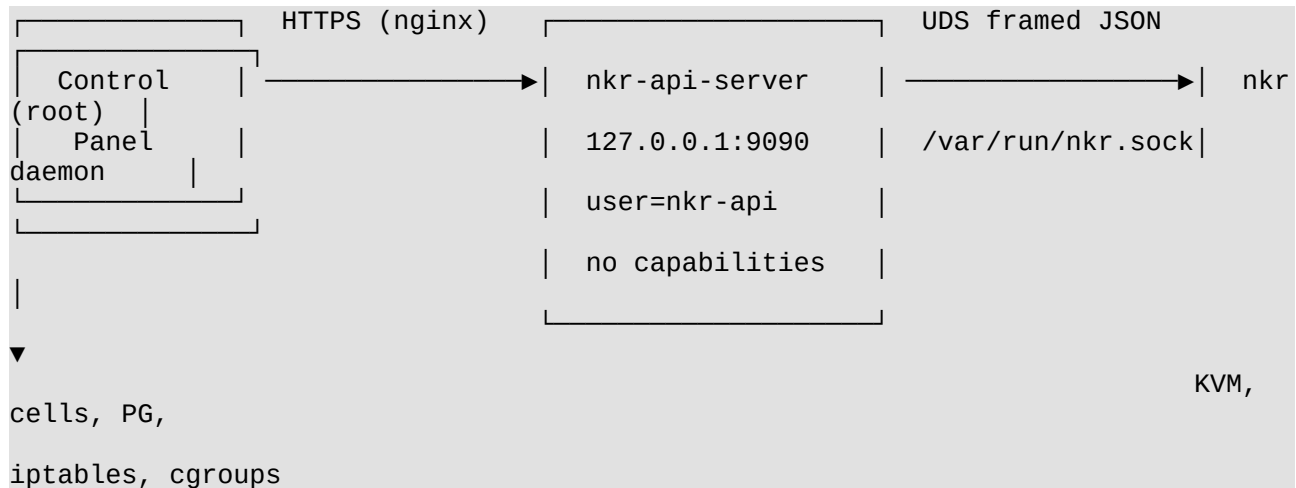
    # Shared nanolinux ELF / bzImage
    # Per-stack CoW snapshots
    # cell_name → cell_id
    # "cell/vm" → vm_id (scoped)
    # Per-cell instance directories

```

Privilege Separation and HTTP API — New in v1.5/v1.6 (extended through v1.6.4)

Starting in v1.5, NKR operates as two cooperating processes with separate responsibilities. The goal is to **move every network-exposed attack surface out of the root process**: a fault in the HTTP parser, in JSON deserialization, or in input validation must not escalate into control over KVM, cgroups, or iptables.

Two-Process Architecture



- **nkr (root daemon)** — `src/main.rs serve`. Only process with hypervisor privileges. Listens exclusively on a *Unix Domain Socket* at `/var/run/nkr.sock` with permissions `0660 root:nkr-api`. Speaks no HTTP and no TCP. Dispatches each `IpRequest` to a typed handler in `src/api.rs`.
- **nkr-api-server (unprivileged proxy)** — `src/bin/nkr_api_server.rs`. Runs as user `nkr-api` (uid created at install time), no capabilities, inside a `systemd` sandbox with `NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome`, `PrivateTmp`, `PrivateDevices`, `RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6`, `MemoryDenyWriteExecute`, and a `SystemCallFilter` that excludes `@privileged @resources @mount @cpu-emulation @debug @obsolete`. The proxy translates HTTP REST → IPC and back. It does not link against `kvm-ioctls`, `vmm`, `cell`, or `state` — the binary weighs ~580 KB vs ~2.2 MB for the daemon (verified).

IPC Wire Protocol

Each request is a *frame* on the UDS: 4-byte *length-prefix* in *little-endian* + JSON body, max 8 MiB. The connection closes after one response — no multiplexing, no conversational state. Read/write timeout per connection is 120 s to cover the longest operation (`POST /instances` with DB clone ~30–40 s).

```
pub enum IpRequest {
    Health,
    ListCells,
    RenderMetrics,
    CreateInstance { cell_hint: Option<String>, body_json: String },
    GetInfo { nkr_name: String },
    DeleteInstance { nkr_name: String, drop_db: bool },
    Action { nkr_name: String, action: String },
    GetLogs { nkr_name: String, tail: Option<usize>,
        from_offset: Option<u64>, max_lines: Option<usize>,
        wait_ms: Option<u64> },
    ModulesAction { nkr_name: String, op: String,
        modules: Vec<String>,
        admin_login: String, admin_password: String },
    CreateDns { nkr_name: String, dns: String, enable_websocket: bool },
```

```

DeleteDns { nkr_name: String, delete_cert: bool },
InitDb { /* ... */ },
PatchConfig { nkr_name: String, body_json: String },
Psql { nkr_name: String, query: String, max_rows: usize },
PurgeCache,
}

```

HTTP Endpoint Catalog

All endpoints (except `/metrics` and `/api/v1/health`) require **Authorization:**

`Bearer $NKR_API_TOKEN` with *constant-time* comparison (`net lock::ct_eq`) to prevent *timing attacks*. Identifier and path validation runs in the proxy and is **re-validated** in the daemon — defense in depth: even an attacker with UDS access cannot inject shell metacharacters or escape outside cell directories.

Method	Path	Function
GET	<code>/api/v1/health</code>	health check (no auth)
GET	<code>/metrics</code>	Prometheus text exposition 0.0.4 (no auth)
GET	<code>/api/v1/cells</code>	lists cells with <code>odoo_version</code> , <code>used_odoos</code> , <code>free_slots</code> , <code>max_odoos</code>
POST	<code>/api/v1/instances</code>	creates tenant (auto cell-selection by <code>odoo_version</code>)
POST	<code>/api/v1/cells/{cell}/instances</code>	creates tenant forcing explicit cell
GET	<code>/api/v1/cells/{cell}/instances/{name}</code>	info + <code>nkr_status</code> (running, <code>pid</code> , <code>ram_mb</code> , <code>uptime_s</code> , <code>port_8069_up</code>)
DELETE	<code>/api/v1/cells/{cell}/instances/{name}?drop_db=1</code>	deletes tenant
POST	<code>/api/v1/cells/{cell}/instances/{name}/actions</code>	<code>{"action": "start\ stop\ restart"}</code>
GET	<code>/api/v1/cells/{cell}/instances/{name}/logs?tail=N</code>	tail / cursor-paginated logs
GET	<code>/api/v1/cells/{cell}/instances/{name}/logs/download</code>	full <code>odoo.log</code> download
POST	<code>/api/v1/cells/{cell}/instances/{name}/dns</code>	provisions/regenerates <code>nginx vhost</code> + Let's Encrypt cert
DELETE	<code>/api/v1/cells/{cell}/instances/{name}/dns</code>	removes <code>vhost</code> (idempotent)

Method	Path	Function
	{name}/dns	
POST	/api/v1/cells/{cell}/instances/{name}/init-db	creates initial DB (async, polled via <code>nkr_status.db_present</code>)
PATCH	/api/v1/cells/{cell}/instances/{name}/config	upserts workers/SMTP into <code>odoo.conf</code>
POST	/api/v1/cells/{cell}/instances/{name}/psql	SQL sandbox: fixed <code>-d db-<tenant></code> , DDL blacklist, audit log
POST	/api/v1/cells/{cell}/instances/{name}/modules/{op}	install/upgrade/uninstall via JSON-RPC
POST	/api/v1/cells/{cell}/instances/{name}/addons/git	clones repo and explodes modules
POST	/api/v1/cells/{cell}/enterprise/git	clones enterprise repo (shared per-cell)
PUT	/api/v1/cells/{cell}/instances/{name}/pylibs	<code>pip install --target=<pylibs/lib></code>
POST	/api/v1/admin/cache/purge	wipes nginx server-side cache

Panel Access Control: IP Whitelist + Per-Panel Token

Access to the HTTP API is locked behind **two independent gates that are enforced together** — an attacker must defeat both, not one:

1. **IP whitelist (perimeter allowlist).** The `nkr-api-server` proxy is never exposed to the open internet. nginx/Caddy sits in front with an explicit allowlist of the authorized panels' IPs (`allow <panel-ip>; deny all;` in the API vhost block). Any origin outside the list gets 403/444 **before** the request ever reaches the proxy's HTTP parser — rate limiting, body-size caps, and the rest of the hardening don't even come into play for unauthorized traffic. The whitelist is the first line: it shrinks the attack surface to a handful of known addresses. *(In the Workspaces use case, the `host` → `panel` webhook mirrors this: the daemon talks to `panel_url` reusing the existing perimeter whitelisting — see §Remote Development Workspaces.)*
2. **Per-panel Bearer token.** Each panel (or each panel environment: production, staging, on-call) carries its **own** `NKR_API_TOKEN`, not one shared global secret. Every endpoint except `/api/v1/health` and `/metrics` requires `Authorization: Bearer <token>`, compared in **constant time** (`netlock::ct_eq`) so the secret never leaks via *timing*.

Because each panel has its own token, **rotating or revoking a compromised panel's access does not affect the others**: you change that one token and restart the proxy, leaving the rest of the fleet untouched. Tokens live in `/etc/nkr/api.env` (root-readable, outside the repo) and never appear in logs (client errors are generic; detail goes only to operator stderr).

Why both together: the whitelist alone does not protect against a pivoted or spoofed panel IP inside the perimeter; the token alone does not protect against brute force or reuse from any internet IP. Requiring IP-on-list **and** valid-per-panel-token means compromising the API requires simultaneously being on an authorized IP *and* holding *that* panel's secret — and even then the attacker is confined to whatever the root daemon re-validates over IPC (no shell, no path traversal, no escaping outside the cells). Real defense in depth, not security theater.

Additional Hardening Applied to the Proxy

- **Default bind 127.0.0.1** — the proxy does not listen on the network by default. nginx/Caddy in front for TLS + the panel IP whitelist described above. Override via `NKR_BIND_ADDR=0.0.0.0` (emits WARN; only for isolated labs).
- **Body size limits** — `POST /instances` 64 KB, `POST /actions` 1 KB, `POST /addons/git` 64 KB, `PUT /pylibs` 256 KB. 413 if exceeded.
- **Bounded log reader** — `read_tail_lines` does reverse seek by 64 KiB chunks with a 4 MiB RAM cap. Rejects DoS via multi-GB `odoo.log`.
- **Bounded thread pool** — `MAX_INFLIGHT=64` concurrent HTTP handlers; excess receives 503 with `Retry-After: 1`. InflightGuard RAII decrements even if the handler panics.
- **Minimal info disclosure** — generic errors to the client (`{"error": "not_found"}`), detail only on operator stderr.
- **Panic protection** — `read_request` uses `?` instead of `unwrap()`; malformed request → 400, no thread crash.
- **verify_dependencies()** at boot — `nkr serve` checks 11 critical binaries (ip, iptables, mount, mkfs.ext4, e2fsck, losetup, chatr, psql, pg_isready, virtiofsd, umount) and logs missing ones before accepting traffic.

Multi-Tenant Operation: The Full Flow — New in v1.6 (extended through v1.6.4)

NKR v1.6 closes the loop between the external panel and the daemon: provisioning a new Odoo customer (with their DNS, cloned DB, admin user with a real password, rate limit, OCA addons, and correct edition) is a sequence of 4 REST calls. v1.6.1 adds the **tier** as a first-class citizen (`tier=production|staging|dev`) with locked profiles for non-prod tiers, and `POST /reload` to refresh Odoo workers without restarting the VM. v1.6.2 adds `POST /balloon` (renews the ACTIVE state of dynamic ballooning) and refines `POST /addons/git` with Double

Hygiene + strict 422 (deterministic mirror of the meta-repo). v1.6.3 empties `dev_mode` in DEV/STAGING (the `REL_OD/HVC0` reload replaces it) and adds the watchdog. v1.6.4 adds HMAC-signed SSO + the `systemouts-addons` internal addons path, async `POST /instances` (202 + create-status polling), `POST /diag` (host-side forensics), and the `/mnt tmpfs` in the `initramfs`. This section describes each piece and the invariants NKR guarantees.

The Canonical Template DB Seed

Each cell has a reserved instance named `<cell>-odoo-template` with `disabled: true` in `compose`. This VM **never boots during operation** — it is only the **source** for clones. The associated DB (`db-<cell>-odoo-template`) is created **once** via Odoo's own `/web/database/create` endpoint, not via `pg_restore` or partial dump:

```
# Procedure (manual, once per cell):
# 1. nkr compose up -d (with disabled:false temporarily)
# 2. Wait for TCP :8069
# 3. POST /web/database/create with master_pwd=admin, lang=es_419,
#    country_code=PE, login=admin, password=admin, demo=False
# 4. pg_dump db-<cell>-odoo-template → /mnt/nkr/templates/<cell>-base-
<ts>.sql.gz
# 5. nkr stop + disabled:true
```

Why this matters: if the template DB is imported from a partial dump, it ends up with `ir_module_module` populated but `ir.asset` empty and some `_register_hook` calls never executed. The first clone of the template, on `/web/login`, tries to compile `web.assets_frontend` with an incoherent asset registry, fails with `Undefined variable: $black` (broken SCSS order), and serializes the error message as valid CSS that gets cached in `ir_attachment` — every subsequent request sees the broken bundle. The reliable way to detect it: count rows in `ir_module_module` `WHERE state='installed'` (must be ≥ 14 , not 7) and verify that a clone's CSS bundle weighs >100 KB and does not contain the string `"A css error occurred"`.

The good template's dump is preserved at `/mnt/nkr/templates/<cell>-base-latest.sql.gz` as a reproducible artifact. If a cell becomes corrupted, restoring from that dump is a minutes-long operation.

POST /instances — Clone + Boot + Admin Password in One Request

The request body:

```
{
  "nkr_name": "client-42",
  "mode": "production",
  "odoo_version": "19.0",
  "edition": "community",
  "workers": 2,
  "admin_passwd": "MasterPwd-xxxxxxx",
  "admin_user_password": "AdminLoginPassword-yyyy",
  "dns": "client-42.example.com"
}
```

NKR processes in order:

1. **Validation** — `nkr_name`, `dns`, `admin_passwd` (charset `[A-Za-z0-9._-]` {16,128}), `admin_user_password` ({8,128}). Re-validated in the daemon.
2. **Cell resolution** — if `cell` is omitted, auto-selects the cell with matching `odoo_version` and lowest `used_odoos`. 409 if none available.
3. **Resource derivation** — `workers=N` automatically yields `chrs=2N+1`, `ram_mb=1024·N`, `limit_memory_soft=400·N` MB, `limit_memory_hard=750·N` MB. For `workers=2`: `ram=2048` MB, `soft=800` MB, `hard=1500` MB (enough to install account with its ~30 dependencies without OOM-killing the worker).
4. **File clone** — `chattr +C` (no-CoW on btrfs) + `cp` of the ext4 rootfs + creation of `var_lib_odoo.ext4` (empty filestore).
5. **DB clone** — `CREATE DATABASE db-<tenant> TEMPLATE db-<cell>-odoo-template` (file-level copy in PG, ~3 s for 80 MB). The admin user inherited from the template has `password=admin`.
6. **Edition opt-in** — if `edition=community` (or null), `append_compose_block` filters the `/mnt/nkr/enterprise/<v>:/mnt/extra-enterprise:ro` line from the generated compose and `rewrite_odoo_conf_full` removes `/mnt/extra-enterprise` from `addons_path`. If `edition=enterprise`, both are kept → tenant sees the manifests of the enterprise repo downloaded via `POST /enterprise/git`. This allows mixing community and enterprise tenants in the same cell.
7. **Append compose block** — the tenant block template inherits from the seed template with three forced overrides: `disabled: false`, `skip_warmup: true`, `logfile = /var/log/odoo/odoo.log`.
8. **compose up -d** — the daemon brings the cell stack up. `db/pgbouncer` services are already alive (idempotent); only the new Odoo starts.
9. **If admin_user_password is present** — NKR polls `:8069` for TCP up (max 120 s) and then runs two JSON-RPC calls against the guest:

```
POST /web/session/authenticate { db, login:"admin", password:"admin" }
POST /web/dataset/call_kw { model:"res.users", method:"change_password",
                           args:["admin", new_password] }
```

On success, returns 201 with the tenant booted and password applied. On failure, 503 `admin_password_setup_failed` with detail — the tenant remains booted but with `admin/admin`. This closes the window where the default password inherited from the template kept working between `POST /instances` and the first panel action.

10. **Persists meta.json** — `<instance_dir>/meta.json` with all original parameters so that `GET /instance/{name}` can reconstruct state.

POST /addons/git — Automatic Multi-Module Explosion

Typical OCA repos contain multiple modules (`account-financial-tools` exports `account_chart_update`, `account_payment_term_aux`, etc., each in its own subdir with its own `__manifest__.py`). Odoo's `addons_path` only scans the first level of each declared path, so cloning the repo to `addons/account-financial-tools/` leaves all modules invisible.

NKR resolves this on the server side:

1. Temporary clone to `addons/.nkr-tmp-<subdir>/`.
2. Layout detection:
 - `tmp/__manifest__.py` → single-module: rename to `addons/<subdir>/`.
 - `tmp/<m1>/__manifest__.py`, `tmp/<m2>/__manifest__.py`, ... → multi-module: each `<m>/` is moved to `addons/<m>/`.
 - No manifests → 422 `no_modules_found`.
3. Each moved module writes a tracker `addons/<m>/.nkr-source` with `repo_url=`, `ref=`, `sha=`. This enables idempotent re-clones over the same repo (legitimate overwrite) and detection of genuine conflicts: if a tenant already had `account_payment` from `OCA/account-payment` and another repo tries to overwrite it, NKR returns 409 `module_conflict { existing_repo, attempted_repo }` without touching anything.
4. `POST /actions {action:"restart"}` (panel responsibility) reloads manifests and the new `ir.module.module` rows show up to install from the UI.

This pattern unifies `addons_path` to a single path (`/mnt/extra-addons`) regardless of the source layout.

HMAC SSO and systemouts-addons — New in v1.6.4

The panel can drop a logged-in operator into any tenant's Odoo backend without ever handling the user's password. `POST /cells/{cell}/instances/{name}/sso {user}` returns a URL `https://<dns>/nkr-sso?`

`u=<login>&exp=<unix_ts>&sig=<hmac_sha256(secret, "u|exp")>` with a 30-second TTL. NKR writes a fresh per-tenant 256-bit HMAC key into `odoo.conf` under section `[nkr_sso]` key `secret` at clone time (`cell.rs::rewrite_odoo_conf_full` — kept out of `[options]` to avoid Odoo's benign unknown option warning; the legacy `nkr_sso_secret` in `[options]` is still honored as a fallback). An Odoo HTTP controller (`auth="none"`) verifies the signature constant-time (`hmac.compare_digest`), checks expiry, looks up the active user, and creates a sudo session — `request.session.session_token = user._compute_session_token(request.session.sid)` — then redirects to `/odoo`. The user's password never leaves the host; compromising the secret means arbitrary login to that one tenant, so rotation = `edit [nkr_sso] secret + POST /actions {restart}`.

Optional defense-in-depth: a `nkr_sso_allowed_referer` filter in `odoo.conf`.

The module that does this — `nkr_sso` — lives in **systemouts-addons**, a cell-level directory (`cells/<cell>/systemouts-addons/`) mounted **read-only** in every instance as `/mnt/systemouts-addons` and inserted in `addons_path` *before* `/mnt/extra-addons` (the tenant's own addons/). Consequences: (1) a client module with the same name as an internal one cannot shadow it (first match wins, and the internal path comes first); (2) `POST /addons/git` never touches `systemouts-addons` → the client cannot see, edit, or overwrite it; (3) one copy per cell, RO — no per-tenant work. `nkr_sso` is pre-installed in each `<cell>-odoo-template` (code in `systemouts-addons/nkr_sso/` + `state=installed` in the template DB) so clones inherit both the code (via the cell-level RO share) and the installed state (via `CREATE DATABASE ... TEMPLATE`); only the secret is regenerated fresh per tenant. The `/mnt/systemouts-addons` mountpoint is baked into the master rootfs (and, since v1.6.4, the `initramfs` mounts a `tmpfs` over `/mnt` so any *future* `/mnt/*` share works without a master rebuild — see §`Initramfs`).

Edge nginx Hardening — Rate Limit and 444

Each tenant vhost generated by `POST /dns` includes:

```
include /etc/nginx/snippets/nkr-hardening.conf;    # 444 over legacy extensions
                                                    # and CMS paths

location ~ ^/web/(login|database|selector|database/manager) {
    limit_req zone=nkr_login_limit burst=5 nodelay;
    limit_req_status 429;
    proxy_pass http://up_<tenant>;
}
```

`nkr-hardening.conf` (shared across all tenants) returns 444 (TCP close with no response) over:

- files ending in `.php`, `.env`, `.sql`, `.bak`, `.yaml`, `.tar`, `.gz`, `.zip` — Odoo serves none of those, they're scanners
- internal paths `/.git/(...)`, `/.svn/(...)` — blocks accidental public-repo discovery
- substrings `wp-admin`, `wp-login`, `MyAdmin`, `phpmyadmin`, `adminer` — well-known CMS tools

444 is preferable to 403/404 because it does not confirm that the server exists (anti-fingerprint) and consumes no bandwidth writing a response.

The `3 r/s burst=5 nodelay` rate limit on login and database manager paths protects against brute-force. It works because Cloudflare runs in DNS-only mode (not proxied), so `$binary_remote_addr` is the real attacker IP. A 10 MB global zone tracks ~160k unique IPs. Bursts up to 5 requests pass instantly (covers the user typing login + remember password); from the 6th onward they get 429. Typical brute-force (10+ requests/s) is throttled at the ~6th attempt.

Server-Side Static Caching

`/web/static/*` (module assets: images, fonts, template CSS) and `/web/assets/<hash>/*` (compiled bundles) are disk-cached at `/var/cache/nginx/nkr_static` with `proxy_cache_path keys_zone=nkr_static:50m max_size=2g inactive=24h`. For caching to work on the tenant's server block—which has `proxy_buffering off` for SSE/long-polling—each cacheable location re-declares `proxy_buffering on` locally.

path	TTL in cache	invalidation
<code>/web/static/*</code>	24 h	POST <code>/admin/cache/purge</code> (fixed URL — when the customer changes the logo the URL doesn't change)
<code>/web/assets/<hash>/*</code>	30 d (Cache-Control: public, immutable)	automatic — <code><hash></code> changes when the bundle changes

POST `/admin/cache/purge` runs `rm -rf /var/cache/nginx/nkr_static/*` from the root daemon and returns `{purged, size_bytes_freed}`. Reconstruction is organic: the next request to a cached asset hits Odoo and re-populates the entry. Acceptable cost: sub-second overhead the first time each asset is requested again.

`nginx -s reload` **does not purge** the cache — files on disk survive reloads, service restarts, and host reboots. Without this endpoint the only alternative was SSH to the host.

sitecustomize.py — Real Client IP in the Odoo Log

By default, `werkzeug` and `gevent.pywsgi` (the server behind `:8072` longpolling) log `REMOTE_ADDR` — the IP of the TCP connection receiving the request. Since `nginx` does `proxy_pass`, that IP is always `10.0.x.1` (the cell gateway), not the client's. `proxy_mode = True` in `odoo.conf` corrects Odoo's internal behavior (sessions, security_groups), but **does not affect the log handlers** of `werkzeug` or `gevent`.

NKR fixes this by injecting a `sitecustomize.py` into `<tenant>-overrides/` that Python loads automatically at startup (via `PYTHONPATH=/tmp/nkr-overrides:$PYTHONPATH` exported by the `initramfs nkr-start.sh`). The script monkey-patches two methods:

```
import werkzeug.serving as _ws
_orig_addr = _ws.WSGIRequestHandler.address_string
def _nkr_addr(self):
    h = getattr(self, 'headers', None)
    if h is not None:
        ip = h.get('X-Real-IP') or (h.get('X-Forwarded-For') or '').split(',')[0].strip()
        if ip: return ip
    return _orig_addr(self)
_ws.WSGIRequestHandler.address_string = _nkr_addr

import gevent.pywsgi as _pw
_orig_fmt = _pw.WSGIHandler.format_request
def _nkr_fmt(self):
```

```

    env = getattr(self, 'environ', None) or {}
    ip = env.get('HTTP_X_REAL_IP') or (env.get('HTTP_X_FORWARDED_FOR') or
''.split(',')[0].strip())
    if ip:
        orig = self.client_address
        try:
            self.client_address = (ip, orig[1] if isinstance(orig, tuple) else
0)
            return _orig_fmt(self)
        finally:
            self.client_address = orig
    return _orig_fmt(self)
_pw.WSGIHandler.format_request = _nkr_fmt

```

The initramfs additionally does `chown odoo:odoo /var/log/odoo` before privilege drop so that Odoo (running as user `odoo` in the guest) can write `logfile = /var/log/odoo/odoo.log` over the RW virtio-fs share. Without that `chown`, Odoo silently falls back to `stdout` and the host never sees `odoo.log`, leaving the `GET /logs` endpoint with empty responses.

Multi-Tenant Deployment

NKR ships a complete deployment toolkit for multi-tenant Odoo 17.

Client Registry

Clients are defined in `deploy/clients.yml`:

```

global:
  pg_ram: 2048
  odoo_ram: 256
  odoo_chrs: 1
  base_disk: /mnt/nkr/images/odoo-base.ext4
  db_statement_timeout: 60000 # ms – max query duration per tenant (v1.1)
  db_conn_limit: 10          # max simultaneous connections per database
(v1.1)

clients:
  - name: acme
    domain: acme.example.com
    db_name: acme_prod
  - name: globex
    domain: globex.example.com
    db_name: globex_prod
    ram: 512 # override
    chrs: 2 # override
    db_conn_limit: 20 # override – heavier-load customer

```

Provisioning Pipeline

```

mt-provision.sh <client-name>
|
|— Create CoW disk:    cp --reflink=auto base.ext4 → <client>.ext4
|— Generate Odoo config: odoo.conf with db_name, dbfilter, workers=2+
|— Generate nginx config: <domain> → 10.0.{cell_id}.<vm_ip>:8069/8072
|— Activate nginx site: ln -s sites-available → sites-enabled

```

```

└─ Reload nginx:          nginx -s reload
└─ PostgreSQL limits (v1.1):
    └─ ALTER DATABASE "<db>" SET statement_timeout = '<N>ms';
    └─ ALTER DATABASE "<db>" CONNECTION LIMIT <N>;

```

Multi-Worker Odoo

Every Odoo instance uses `workers = 2+` (drops the werkzeug single-thread mode):

- :8069 — synchronous HTTP workers
- :8072 — gevent for long-polling and WebSockets

Hot Module Update

The `deploy/update.sh` script provides near-zero-downtime module updates:

Mode	Command	Downtime
Production	<code>update.sh</code>	~2s (clean shutdown via <code>hvc0</code> + restart)
Test	<code>update.sh --test</code>	0 (runs on port 8070)
Rollback	<code>update.sh --rollback</code>	~2s
DB upgrade	<code>update.sh --update-db</code>	~30 seconds

Update flow:

1. Automatic backup of current modules (keeps last 5)
2. Stop Odoo VM via `nkr stop` (SIGTERM → `hvc0 SHUTDOWN` → clean exit ~2s; PostgreSQL keeps running)
3. Mount disk, rsync modules with `__manifest__.py`
4. Restart Odoo VM via `nkr restart` or `compose`

Target Architecture

Server (16–32 GB RAM), 5 cells × (1 PG + 1 PgBouncer + 20 Odoos)

```

└─ Cell 1 "empresa_prueba" — nkr-br1, 10.0.1.0/24
    └─ VM empresa_prueba-pg          (id=1, 10.0.1.2, 2GB RAM)
    └─ VM empresa_prueba-pgbouncer   (id=2, 10.0.1.3, 128MB RAM)
    └─ VM empresa_prueba-odoo-01     (id=3, 10.0.1.4, 256MB RAM)
    └─ ... empresa_prueba-odoo-20    (id=22, 10.0.1.23, 256MB RAM)

└─ Cell 2 "cafeteria" — nkr-br2, 10.0.2.0/24
    └─ ... (same structure)

└─ nginx (on host) — SNI map → cell IP:8069/8072
└─ Exposed ports: 80, 443, SSH
    Everything else: internal on the per-cell bridge

```

Density scaling (32 GB server):

Scenario	RAM/Instance	Instances in 32 GB
v1.1 base	~640 MB	50
v1.2 + PMEM	~440 MB	72
v1.3 + VirtIO-FS DAX	~370 MB	~85
v1.4+ all features + Balloon	~310 MB	~100

*Effective density comes from three coordinated mechanisms: VirtIO-FS+DAX shares code (binaries, libs, .pyc) across VMs by reading from the same host backing file; VirtIO-PMEM+DAX eliminates the RO rootfs copy in the guest page cache; VirtIO-Balloon reclaims idle guest RAM back to the host via MADV_DONTNEED. **KSM does not apply with this layout** (see §VirtIO-Balloon — the memFd+MAP_SHARED required by vhost-user is incompatible with MADV_MERGEABLE).*

Technical Note on Density Metrics. Our current measurements show a projected consumption of **70–90 GB for 100 instances under active load**, representing a 55% saving over traditional Docker setups. The target of **100+ instances in 32 GB RAM** is an *Optimized High-Density* scenario reachable when guest working sets are mostly idle: it is achieved by leveraging VirtIO-Balloon to reclaim idle guest RAM and VirtIO-FS with DAX to share code binaries and .pyc files across VMs, effectively reducing the per-tenant footprint to ~310 MB. Both numbers are valid; they describe the same system under different load assumptions.

Density, Storage, and Database: Post-Phase-0 Lessons — v1.6.5–v1.6.9

After stabilizing the multi-tenant loop, a telemetry "Phase 0" measured NKR's **actual** behavior on the provisioned hardware (a 64 GB host, 21 running VMs, KSM=0) before investing in speculative optimizations. The measurements redirected several engineering decisions.

Measured PSS Baseline

Real per-VM consumption, measured with PSS (*Proportional Set Size* — it splits shared memory across the processes that map it; the honest metric for density):

Profile	PSS
idle dev	223 MB (Shared DAX 69 MB + Private 191 MB)
idle staging	197 MB
prod workers=2	343 MB
dev under real traffic	524 MB

The central finding: **the DAX-mode rootfs alone already contributes ~31% of the sharing**, with no additional technologies. This confirmed that NKR's density does not depend on KSM (rejected

by the `memfd+MAP_SHARED` layout, see §VirtIO-Balloon) but on DAX + VirtIO-FS — and closed the topic: KSM is not revived even when the arithmetic suggests it.

Symlinking the Infrastructure Rootfs to the Immutable Master

The per-cell PostgreSQL and PgBouncer rootfs (`cells/<cell>/{postgres,pgbouncer}-root.ext4`) became **symlinks to the immutable master** (`/mnt/nkr/images/{postgres,pgbouncer}.ext4`), just like Odoo tenants. Validated: the rootfs is not written at runtime (PG's `datadir` lives in a separate share, PgBouncer's config comes from an override share, logs go to another share). With DAX + symlink, the rootfs page cache is physically shared across cells (e.g. v17 and v19). Measured savings: **~400 MB of RSS** (from 760 MB → 363 MB for the 4 infrastructure VMs) + **914 MB on disk**.

Initramfs Optimization: Monolithic Kernel

Two cuts reduced the decompressed initramfs per VM from **4.20 MB** → **1.17 MB** (−72%):

1. The Rust `nkr-watcher` binary was removed from the *embedding* (~400 KB/VM) — it was never used in production; the real watcher is the busybox subshell of the init script.
2. The `lib/modules/*.ko` were removed from the generated initramfs (~3 MB/VM) — the custom kernel is **monolithic** (`--disable MODULES` at build), with all drivers built in (`ext4`, `jbd2`, `virtio_blk`, `virtio_net`, `virtio_console`, `virtio_fs`, `virtio_pmem`, `virtio_balloon`). The init script calls neither `modprobe` nor `insmod`.

With 21 active VMs, this freed **~63 MB** on the host.

Snapshot/Restore: Evaluated and Dropped

The whitepaper's original target ("100 Odoos in 32 GB") assumed hardware different from the actual host (64 GB). A 3–4 week snapshot/restore refactor was evaluated in Phase 0 and **dropped**: it would have yielded ~25–75 MB extra per idle VM and ~0 under traffic — not worth the cost or complexity. Early telemetry prevented technical debt; the case is documented as an internal study.

Per-Tier PostgreSQL Connection — v1.6.9

A chronic hang of dev/staging tenants (it left them unresponsive on `:8069`, triggering the watchdog in a loop) was diagnosed —with the pre-restart forensics, see §Watchdog and Forensics — as **Odoo connection-pool exhaustion** (`psycopg2.pool.PoolError: The Connection Pool Is Full`), reproduced deterministically on a test tenant. It was not a kernel D-state, nor memory, nor VirtIO-FS. The cause: `workers=0` (threaded) against PgBouncer in *transaction mode* pins/queues connections → Odoo's internal pool fills and never recovers → `:8069` returns 500 → watchdog restart loop.

DB routing and `db_maxconn` became **per-tier**, applied by the daemon when generating `odoo.conf` on each clone:

Tier	DB target	db_maxconn	Why
dev / staging	direct PG 10.0.{cell}.2:5432	10	threaded + PgBouncer transaction mode = pinning → pool full. Direct removes the mismatch; db_maxconn=10 throttles and reuses warm connections. No pooler needed (single-tenant, low traffic).
prod	PgBouncer 10.0.{cell}.3:6432	15	prefork (workers≥1): each worker = one process serving one request at a time → bounded concurrency → PgBouncer transaction mode multiplexes the N workers.

The per-cell PgBouncer (today used only by prod) was sized for the cap of 7 prod projects/cell: max_client_conn=250, default_pool_size=20, reserve_pool_size=5.

Validated: direct PG + db_maxconn=10 → **0 PoolError** and ~28 ms under realistic load and extreme burst (vs a permanent hang with PgBouncer + a large pool).

Watchdog and Pre-Restart Forensics — v1.6.8, enabled

The watchdog (watchdog.rs, a daemon thread) TCP-probes :8069 per running tenant every 15s; after HUNG_THRESHOLD_SECS=120 consecutive misses it triggers an automatic restart. As of v1.6.8:

- The threshold was raised from 60s to **120s** after observing false positives during legitimate deploys (a REL_OD with live websockets + running cron could take >60s).
- It became **REL_OD-aware**: after a reload, during a RELOAD_GRACE_SECS=240 grace window the effective threshold rises to RELOAD_THRESHOLD_SECS=180 — covering the deploy worst case without delaying detection of real hangs on idle VMs.
- Before restarting, capture_hang_forensics dumps to /mnt/nkr/hangs/<name>-<ts>.txt: per-thread stacks/wchan/state of the VMM process (vcpus in D give away the stall), the cgroup's memory {current, max, events} + PSI {memory, cpu, io}.pressure (distinguishes reclaim-thrashing from deadlock), - ram vs memory.max (catches first-boot under-provisioning), and the guest's internal RAM. It is best-effort and never blocks the restart.

These forensics are what let us rule out memory/VirtIO-FS/D-state and point at the DB pool (see above). The watchdog **ships enabled** (validated by recovering a real hang); bypass with

NKR_WATCHDOG_DISABLED=1.

Observability and Metrics

NKR ships a low-level telemetry system implemented in the hypervisor itself that measures and exposes resources used in real time by every micro-VM, with no need to deploy agents inside the *guests*. Today these metrics are **host-side** — what each VM costs the host (VMM process RSS, VMM CPU time, TAP bytes, VMM block I/O). Guest-internal metrics (RAM/CPU/disk *as seen inside* the tenant — useful for showing the client "your Odoo uses X of Y RAM") are designed but not yet implemented; see *Roadmap*.

The metrics engine extracts information via lightweight probes from `procfs` and the host network subsystem:

- **CPU%:** ~50 ms synchronous sampling window over `/proc/{pid}/stat` (200 ms for the `nkr stats CLI`). Jittery by nature of the short window — average it in Grafana for dashboards, or derive a counter (`nkr_guest_cpu_seconds_total`, planned).
- **RAM (VmRSS):** Physical RSS from `/proc/{pid}/status`. The key density metric — real host memory the VM costs *now*, vs. RAM pre-allocated to the VM.
- **Balloon:** MB currently inflated in the VirtIO-Balloon (= returned to the host). Reflects the **runtime** state — 0 when ACTIVE, e.g. 256 when a DEV tenant has decayed to IDLE. Updated on every ACTIVE ↔ IDLE transition (v1.6.4).
- **DAX savings:** estimated RAM not duplicated as guest page cache (`max(0, ram_allocated - rss - balloon - 50 MB overhead)`) for VMs with a DAX path (`virtio-fs/pmem` rootfs).
- **Disk (I/O):** Cumulative read/write bytes from `/proc/{pid}/io`. *Caveat:* for Odoo tenants the rootfs is `virtio-fs` served by a separate `virtiofsd` process → those reads are not seen by the VMM; this counter only covers the block device (`/var/lib/odoo`), so it's usually low or 0.
- **Network (TAP):** TX/RX volumetric throughput on the TAP interface using `/proc/net/dev`.
- **KSM status:** read from `/sys/kernel/mm/ksm/` for operational visibility. In v1.4+ this metric reports 0 MB by design: the `memfd+MAP_SHARED` layout required by `vhost-user` is incompatible with `MADV_MERGEABLE` and the kernel rejects the merge. The reading is preserved to detect re-activation if a hybrid memory layout is ever introduced.

```
sudo nkr stats # all VMs
sudo nkr stats empresa_prueba-odoo-01 # filtered by name/hash/id
```

Native Prometheus Exporter — New in v1.1

GET `/metrics` on `:9090` (served by `nkr-api-server`, **no auth** — meant for direct Prometheus scrape; put it behind a private network or an IP allowlist), `text/plain`;

version=0.0.4 exposition. Each scrape takes a ~50 ms CPU window. Implemented with `std::net::TcpListener` + a string builder, no extra crates.

Exposed metrics (all per-VM ones carry `vm="<nkr_name>"`):

Metric	Type	Description
<code>nkr_cpu_pct{vm}</code>	Gauge	CPU percentage of the VMM process (~50 ms window) — jittery; average in Grafana
<code>nkr_rss_mb{vm}</code>	Gauge	Real physical RAM (RSS) of the VMM process, in MB
<code>nkr_ram_allocated_mb{vm}</code>	Gauge	RAM assigned to the VM at boot (<code>compose.ram</code>)
<code>nkr_balloon_mb{vm}</code>	Gauge	RAM inflated in the VirtIO-Balloon — runtime value (0 ACTIVE / e.g. 256 IDLE)
<code>nkr_dax_savings_mb{vm}</code>	Gauge	Estimated RAM saved by DAX/virtio-pmem (no page-cache duplication)
<code>nkr_total_savings_mb{vm}</code>	Gauge	<code>balloon_mb + dax_savings_mb</code>
<code>nkr_io_read_bytes{vm}</code> / <code>nkr_io_write_bytes{vm}</code>	Counter	Block-device bytes read/written by the VMM (cumulative) — see virtio-fs caveat above
<code>nkr_net_rx_bytes{vm}</code> / <code>nkr_net_tx_bytes{vm}</code>	Counter	TAP bytes received/sent (cumulative)
<code>nkr_ksm_savings_mb</code>	Gauge	MB saved globally by KSM (in v1.4+ always 0; see §VirtIO-Balloon)

Added in v1.6.4: `nkr_cpu_seconds_total{vm}` / `nkr_cpu_throttled_seconds_total{vm}` (counters from the VM's cgroup `cpu.stat` — supersede the jittery `nkr_cpu_pct`), `nkr_cgroup_memory_bytes{vm}` (from `memory.current`), `nkr_up{vm, cell, tier}` (1/0, includes stopped tenants — an info metric: `metric * on(vm) group_left(cell, tier) nkr_up`), `nkr_build_info{version}`, and cluster totals (`nkr_vm_count`, `nkr_total_{rss, balloon, dax_savings}_mb`).

Per-instance JSON endpoint (v1.6.4): GET

`/api/v1/cells/{cell}/instances/{name}/metrics` returns a JSON snapshot for *one* VM (cgroup CPU/mem, balloon, DAX savings, RSS, net, IO, a `disk` array via host-side `du/stat`, `cell/tier`, uptime, `chrs`, `as_of/stale`). The daemon caches each VM's result ~30s (the disk `du` ~5 min) so a panel polling every 30–60s while a "Metrics" tab is open costs essentially nothing — the cache is the rate limit (no 429s). This is what the SaaS panel uses for the per-tenant

metrics view; `/metrics` (Prometheus) stays for a future Grafana. Per-VM disk is deliberately *not* in `/metrics` (a du per scrape $\times$ 100+ VMs would be O(seconds)).

Also added in v1.6.4 — guest-internal RAM:

`nkr_guest_mem_total/available/free/cached_bytes{vm}` (and `guest_mem` in the per-instance JSON) — RAM *as the guest sees it* (`MemTotal/MemAvailable/MemFree/Cached`), via the virtio-balloon stats virtqueue (the device's 3rd queue, `VIRTIO_BALLOON_F_STATS_VQ`; the vmm drains it ~every 30s and persists the snapshot to the VM state file). This is what the panel shows the client as "your Odoo uses X of Y RAM".

Not yet exposed (ops-only, future): host-level metrics (`/proc/meminfo`, `/proc/stat`, `statvfs` → "how much RAM is left on the box").

Remote Development Workspaces (Dev-Cells) — New

NKR hosts more than production Odoo tenants. The same machinery — KVM micro-VMs, btrfs reflink, virtio-fs, a minimalist initramfs, ballooning, the file-based control watcher — powers a second workload class: **Dev-Cells**, ephemeral remote development environments, one per developer, reachable from the browser or from VS Code Desktop over an outbound tunnel. A developer requests a workspace from the panel, and ~30 seconds later they have a micro-VM with a fresh copy of their client's DB, their repo cloned, their persistent profile mounted, and a VS Code editor connected — all without opening a single inbound port on the host.

The design thesis is the same as the rest of NKR: **reuse 70–80% of the existing stack and build only what is new**. Reused: virtio-fs for RW shares, btrfs reflink for O(1) cloning, the initramfs init scripts, dynamic ballooning, the file-based control watcher, the `activity_lock`. What is new is a tenant class (`dev-workspace`) with its own profile, a tunnel bootstrap pipeline, and per-developer persistent-profile and secret management.

Code Isolation: One Daemon, Separate Module

One architectural directive governs everything below: **a single daemon, isolated code**. Workspaces are NOT a second binary or a fork of the Odoo flow. They live in `workspace_*` modules (`workspace_orchestrator`, `workspace_spawn`, `workspace_network`, `workspace_initramfs`, `workspace_inotify`, `workspace_panel`, ...) and the proxy's HTTP router only gains a handful of new routes. In particular, **the Odoo flow's `cell::Tier` enum is left untouched**: `WorkspaceResources` is an independent type inside the module, which avoids editing the dozens of sites that branch on Odoo tenant tier. NKR's Law #1 — *don't break what already works* — holds by construction: a failing Dev-Cell cannot take down a production tenant, because they share the daemon but not the orchestration code.

The Four Pillars

Pillar 1 — DB Hydration: golden PGDATA + reflink (full isolation)

Each Dev-Cell runs **its own PostgreSQL inside the VM** — no production cell's PG engine is shared. Compute isolation is non-negotiable. The trick to keep this "boot in seconds" is the same one behind NKR's density: **btrfs reflink**.

A cell-level artifact is kept, `images/pgdata-golden-<v>.ext4` — a pre-hydrated PGDATA, periodically refreshed from an anonymized production snapshot. When a Dev-Cell is created, the daemon does `cp --reflink=always` of that golden into the instance's `pgdata.ext4`: **O(1), zero initial physical copy**, pages diverge only on-write. An empirical smoke test measured **205 MB physical for 4 × 200 MB nominal** reflinked PGDATAs. The Dev-Cell's PostgreSQL ships preinstalled in the master rootfs (binaries + init, no data); the data always arrives via reflink.

Two alternatives were evaluated and rejected: cloning via `CREATE DATABASE ... TEMPLATE` against a shared PG (would break full isolation) and `pg_dump/pg_restore` at boot (5–60 s, burns IOPS every boot, breaks "seconds"). Reflink wins on all three axes.

Pillar 2 — Persistent Developer Profile

The Dev-Cell is **cattle, not a pet**: the VM is ephemeral, but the developer's *work* survives. This is achieved with an RW virtio-fs share mounting the host's `/opt/nkr/dev-profiles/<user>/home` onto the guest's `/home/dev`:

```
shares:
- /opt/nkr/dev-profiles/empresa_prueba-alice/home:/home/dev:rw
- /opt/nkr/dev-profiles/empresa_prueba-alice/.ssh:/home/dev/.ssh:rw
```

Everything the developer leaves in `$HOME` — `~/vscode-server`, `~/config`, `~/ssh`, `~/local` (where `pip install --user` and `npm install` land), their cloned personal repo — **persists across VM destructions**, because it lives on the host. When the VM is recreated the next day, the profile is already there.

The only non-trivial point is UID/GID mapping: `virtiofsd` respects host UIDs by default, so all Dev-Cells run a fixed `dev` user with `UID=1000 GID=1000`, and the daemon creates `dev-profiles/<user>/` with those same IDs at provision time (modes 755 for `home`, 700 for `.ssh/.anthropic`). 1:1 mapping, no permission friction. *Privacy*: profile files are readable by root on the host — this is documented explicitly to the developer (don't store personal secrets outside the intended channels).

Pillar 3 — Bootstrap and Silent Pipeline

Boot is a deterministic pipeline with a *state machine* and *fail-fast*. The guest → host communication channel is the same pattern as NKR's control watcher: **virtio-fs, not virtio-console** — the guest writes status files to a share the host reads with `inotify`.

```
[1] Host: POST /api/v1/workspaces (panel)
    └─ daemon: reflink golden PGDATA + write control env file + spawn the VM

[2] Guest boot (standard NKR initramfs, dynamic virtio-fs slot parsing):
```

- └ mounts rootfs RO, profile RW (/home/dev), control share
- └ init reads /tmp/nkr-env from the control share:
 - ANTHROPIC_API_KEY=... (per-developer key, see Security)
 - GIT_REPO=... GIT_BRANCH=...
 - DEV_USER=empresa_prueba-alice
- └ exec /usr/local/bin/dev-cell-bootstrap

[3] dev-cell-bootstrap (JSON state machine, fail-fast):

- └ injects env vars into /etc/environment
- └ clones the repo (or fetch+checkout if the profile already had it)
- └ starts local PostgreSQL (on the reflinked PGDATA)
- └ starts the sandbox tenant's Odoo
- └ as user dev: `code tunnel --accept-server-license-terms --name <tunnel>`

[4] code tunnel prints its URL; the wrapper WRITES it to a virtio-fs share file visible to the host (next to the state machine's .nkr-bootstrap-status)

[5] Host:

- └ inotify (IN_MODIFY) on the workspace's log dir detects the change
- └ persists tunnel_url + phase into the state file
- └ webhook to the panel (single attempt, short timeout) → panel shows "Open in VS Code"

Each phase (Provisioning → Booting → Tunnel-login-required → Ready, or Failed) is materialized in .nkr-bootstrap-status. If a phase fails (e.g. postgres_start_failed from a missing golden), the host detects phase=failed and triggers the **guillotine**: it destroys the VM in ~15 s and preserves the profile. *Stateless host*: the daemon **does not queue events to disk**. If the webhook to the panel fails (timeout, 5xx, network blip), the event is **dropped** — no retries, no queue. .nkr-bootstrap-status is the single source of truth; the panel resyncs with GET /workspaces/{name}/status once it regains connectivity.

Pillar 4 — Resource Containment

The dev-workspace profile is an independent WorkspaceResources (not an Odoo Tier):

Component	Working set	Notes
Base OS (kernel + busybox + supervisor)	~50 MB	minimalist initramfs
Local PostgreSQL	~200 MB	shared_buffers=128M, max_connections=20
Odoo workers=1 (prefork)	~500–700 MB	the bottleneck is the Odoo lib
code tunnel daemon	~80–120 MB	persistent connection + relay
VS Code Server (dev connected)	~150–300 MB	grows with extensions/language servers
Peak buffer (compiling modules / pip install)	~200 MB	
Recommended total	~1.5 GB	

```
WorkspaceResources {
  workers: 1, chrs: 2, ram_mb: 1500,
```

```

    limit_memory_soft: 1000 * MB, limit_memory_hard: 1200 * MB,
    balloon_mb: 0,           // ACTIVE at boot (devs are active)
    balloon_idle_mb: 512,    // IDLE squeeze ≥ 512 MB (avoids idle VS Code Server
OOM)
}

```

The **safe lower bound is 1300 MB** — below it OOM-kills appear when compiling custom modules, running `pip install` of heavy packages (numpy/pandas), or loading a large workspace with the TypeScript language server. On a ~128 GB host with ~~21 PROD Odoo tenants running~~ (50 GB), the **proposed cap is 30 active Dev-Cells** ($30 \times 1.5 \text{ GB} = 45 \text{ GB}$), leaving ~33 GB of headroom for OS, page cache, and peaks.

Networking: Isolated Subnet, Default DROP

Dev-Cells **do not share the Odoo tenants' subnet**. They live in their own `10.99.0.0/24` subnet hanging off a dedicated `nkr-devbr0` bridge, with a default `iptables DROP` policy: a Dev-Cell cannot reach another or the production stacks. The only allowed traffic is the outbound the tunnel needs (editor relay) and the Dev-Cell's access to its own local PostgreSQL (which lives *inside* the VM, not in another). The per-bridge L2/L3 isolation NKR already applies to cells is reused as-is.

Workspaces API

The proxy gains a minimal route group ("minimal router hooks" directive). All require the same access control as the rest of the API — **panel IP whitelist + per-panel Bearer token** (see §Panel Access Control):

Method	Path	Function
POST	<code>/api/v1/workspaces</code>	creates a Dev-Cell: golden reflink + register + cap enforcement. Body: <code>{name, user, git_repo?, git_branch?, anthropic_api_key?, editor_target?}</code> . Async → 202 <code>{name, phase, status}</code>
GET	<code>/api/v1/workspaces/{name}/status</code>	reads <code>.nkr-bootstrap-status</code> and returns it (phase, tunnel_url) — so the panel can reconcile
POST	<code>/api/v1/workspaces/{name}/rebuild-db</code>	stop → <code>rm pgdata.ext4</code> → fresh golden reflink → start. ~5–15 s. Idempotent via <code>activity_lock</code> (concurrent second call → 409 <code>activity_in_progress</code>)
DELETE	<code>/api/v1/workspaces/{name}</code>	manual tear-down: SIGTERM the VMM, cleanup

Method	Path	Function
		TAP/network, preserve the profile
POST	/api/v1/workspaces/_gc	garbage-collect (invoked by the timer; see below)

The `editor_target` field decides the URL emitted once the tunnel is ready:

```
# POST /api/v1/workspaces → 202 {"name":"dev-alice-empresa_prueba-a2","phase":"Provisioning"}
# Then: GET /api/v1/workspaces/dev-alice-empresa_prueba-a2/status until phase=ready

# editor_target=desktop → opens the dev's VS Code Desktop:
#   vscode://vscode.remote-server/tunnel+nkr-dev-alice-empresa_prueba-a2/home/dev/workspace/repo
# editor_target=web (default) → opens in the browser:
#   https://vscode.dev/tunnel/nkr-dev-alice-empresa_prueba-a2/home/dev/workspace/repo
```

The **DB rebuild is developer-controlled**, not a cron: NKR runs the reflink when it receives the authenticated call and **does not implement rate-limit** — the limit ("1 rebuild/user/day" or whatever) is the panel's responsibility. Same philosophy as the rest of the API: NKR provides the mechanical primitive, the panel defines policy.

Workspaces Security

- **Host → panel webhook: existing token + IP whitelisting.** No mTLS, no new architecture. The daemon notifies the panel (`panel_url` in `/etc/nkr/panel.conf`) reusing the **same Bearer token** that already governs the API and the existing **perimeter whitelisting**. It is the exact mirror of the panel access control described in §Panel Access Control: panel → NKR goes with `whitelist+token`, and NKR → panel does too.
- **Secrets without a shared master key.** The `ANTHROPIC_API_KEY` is **per-developer**, read by the host from `/opt/nkr/dev-profiles/<user>/.anthropic/key` and injected as an env var at bootstrap. Compromising a Dev-Cell exposes *only* that developer's key, never a master key that would open all of them. Each key carries a monthly budget managed by the panel. An internal proxy that rewrites `x-api-key` so the real key never touches the guest is noted as a Phase 2 improvement.
- **code tunnel is purely outbound.** It opens an OUTBOUND connection to the editor's infrastructure; it **exposes no inbound ports** on the host (zero firewall changes). The documented trade-off: code transits the editor's relay — relevant if the repo carries client-side secrets.
- **No inbound SSH.** The VS Code terminal (a PTY inside the Dev-Cell) covers `tmux/htop/debugging`. No SSH is opened to the guest.

Lifecycle: Ephemeral by Design (GC)

Dev-Cells are cattle, not pets. Two destruction triggers, **no exceptions**:

1. **Inactivity**: 12 h with no tunnel network traffic → destroy.
2. **Daily 04:00 (Lima time, UTC-5)**: a systemd timer (`OnCalendar=*-*-* 04:00:00 America/Lima`) invokes `nkr workspace-gc --all`, which iterates `cells/*/instances/<dev-workspace-*>` and destroys **all** of them — silently, no wall, no grace period. The advance warning to the developer (~30 min before) is the panel's responsibility.

In both cases **the profile persists**: only the VM and its reflinked `pgdata.ext4` are destroyed. The developer comes back the next day, requests a new workspace, and finds their `~/ .vscode-server`, `~/ .config`, `~/ .ssh`, repo, and packages intact. Philosophy: the VM is disposable, the work is not.

Comparison with Existing Solutions

NKR vs Docker

Dimension	Docker	NKR
Isolation	Shared kernel (namespaces + cgroups)	Full VM (KVM, separate kernel)
Kernel vulnerability	Affects all containers	Affects only the VM with that kernel
CPU guarantee	cgroup <i>shares</i> (soft limit)	Core pinning + cgroupv2 (hard limit)
RAM	<i>Overcommit</i> by default	Exclusive, no overcommit
Binary size	<code>dockerd</code> ~100 MB + <code>containerd</code> + <code>runc</code>	~2–4 MB single binary
Boot time	~1–3 s (process start)	~1–2 s (VM boot)
Restart time	~3–5 s	~2 s (clean <code>hvc0</code> shutdown)
Disk format	Layered overlay filesystem	Raw ext4 (CoW snapshots)
Network	veth + bridge	TAP + per-cell bridge + iptables
Multi-stack	Manual compose per stack	<code>nkr cell</code> with isolated subnets

NKR vs Firecracker

Dimension	Firecracker	NKR
Language	Rust	Rust

Dimension	Firecracker	NKR
KVM interface	Direct (kvm-ioctls)	Direct (kvm-ioctls)
VirtIO	MMIO	MMIO
Focus	Serverless (AWS Lambda)	Multi-tenant SaaS (Odoo)
Disk management	External	Integrated (nkr pull/build, OCI → ext4)
Orchestration	None (external: containerd)	Integrated (nkr compose, nkr cell)
MT tooling	None	Complete (Cell System, instance cloning)
Volume injection	External	Integrated (pre-boot mount + VirtIO-FS)
CPU model	Standard vCPU	"Chrs" (20% granularity with pinning)
Shutdown	Kill process	Coordinated VirtIO-Console (~2s)
Lines of code	~70,000+	~16,000 (focused scope)

NKR vs QEMU/KVM

Dimension	QEMU/KVM	NKR
Binary size	~20–50 MB	2–4 MB
Device model	Full x86 emulation (PCI, USB, ACPI...)	Minimal VirtIO-MMIO only
Configuration	Complex CLI / libvirt XML	Simple CLI flags / YAML
Boot time	~3–10 s	~1–2 s
Dependencies	libvirt, qemu, virt-manager	None (just /dev/kvm)
Attack surface	Large (full emulation)	Minimal (6 MMIO device types)

NKR vs Odoo.sh (PaaS)

Dimension	Odoo.sh (PaaS)	NKR (Private Cloud)
Isolation model	Shared containers	Hardware-isolated micro-VMs
Infra control	Managed (black box)	Full control (kernel / OS / network)
Memory density	Limited by PaaS architecture	Ultra-high (DAX + ballooning)
Multi-version deploy	Complex across projects	Native via Cell System
I/O latency	Variable (public cloud)	Predictable (NVMe mirror +

Dimension	Odoo.sh (PaaS)	NKR (Private Cloud)
		io_uring)

Security Model

Isolation Boundaries

Layer	Mechanism
CPU	KVM hardware virtualization (VT-x/AMD-V). The guest runs in ring 0 of a separate address space.
Memory	GuestMemoryMmap creates dedicated memory regions. No memory shared across VMs.
Disk	Each VM has its own ext4 file. No shared overlay filesystem.
Network	Separate TAP device per VM. Per-cell L2 bridge. Per-VM iptables rules. L2 ebtables rules (v1.1): only the assigned MAC+IP can emit traffic.
Process	Every VM runs as a separate host process. SIGTERM → hvc0 → clean shutdown. Zombie state detected via /proc/pid/status.
Syscalls	Seccomp BPF <i>jailer</i> (v1.2) restricts the vCPU process to ≤31 allowed syscalls after init.

Attack Surface

NKR's attack surface is significantly smaller than Docker's or QEMU's:

- **No userspace device emulation** (vs QEMU): only native MMIO handlers (net, block, VirtIO-FS, Balloon, PMEM, Console + serial)
- **No shared kernel** (vs Docker): a guest kernel exploit doesn't affect the host
- **No container-escape paths**: no namespaces, cgroups, or procfs sharing
- **Minimal host interaction**: only file I/O (disk/mmap), TAP read/write (network), and serial output
- **L2 isolation** (v1.1): ebtables rules prevent IP/MAC spoofing between tenant VMs on the bridge
- **Per-cell L3 isolation** (v1.3): per-cell subnets; inter-cell routing is not enabled by default

- **Privilege separation** (v1.5): the HTTP frontend (parser, JSON deserialization, input validation) runs in an unprivileged process with no capabilities. An RCE in the proxy does not compromise KVM/cgroups/iptables — the attacker is confined to whatever the daemon allows over IPC, all re-validated in the daemon
- **Double validation** (v1.6): identifiers (`nkr_name`, `cell`, `dns`) and paths (`addons_path`) pass a whitelist regex both in the proxy and in the daemon; YAML/shell/path-traversal injection blocked at both edges
- **Edge nginx** (v1.6): rate limit using real `$binary_remote_addr` (Cloudflare DNS-only), 444 close over CMS/legacy paths + `.git/` `.svn` dirs, server-side static cache to reduce hits to Python workers

Seccomp BPF *Jailer* — New in v1.2

Implementation: `seccomp.rs` — ~170 lines

Before entering the vCPU run loop, NKR installs a `SECCOMP_MODE_FILTER` program built at runtime from a static allowlist of 31 syscalls. The filter uses `libc::prctl` directly, with no extra dependencies.

- **Preamble:** `prctl(PR_SET_NO_NEW_PRIVS, 1)` (kernel-required before installing the filter)
- **Policy:** `SECCOMP_RET_KILL_PROCESS` for any syscall outside the allowlist
- **Allowlist includes:** `read`, `write`, `ioctl` (KVM ioctls), `mmap`, `madvise`, `clone` (`thread::spawn`), `futex`, `io_uring_*`, `epoll_*`, `eventfd2`, `openat`, `pread64/pwrite64`, `clock_gettime`, `exit_group`, and `stdlib` essentials
- **Timing:** Installed *after* `VirtioNetDevice::new()` (which spawns the RX thread)
- **Fallback:** If `prctl` fails (kernel < 3.17 or denied permissions), NKR emits a warning and continues without the filter

Operational Security

- Only ports 80, 443, and SSH (configurable) are exposed externally
 - **The control API is never published open:** it sits behind an **IP whitelist** (only the authorized panels' addresses reach the proxy) + a **per-panel Bearer token** (constant-time comparison; rotating one does not affect the rest) — both gates enforced together (see §Panel Access Control)
 - All inter-VM traffic is confined to the per-cell bridge
 - Requires root for KVM/TAP/iptables (intentional — no rootless mode)
 - The Seccomp filter (v1.2) restricts the vCPU process to the minimal syscall footprint
-

Limitations and Future Work

Current Limitations

Limitation	Impact	Planned Resolution
Single vCPU per VM	No SMP in guests	Multi-vCPU support (medium priority)
VirtIO-MMIO only	No PCI passthrough	Enough for target workloads
VirtIO-FS bound to vhost-user	Needs external virtiofsd daemon	Setup automation in a future version
PMEM requires guest kernel support	Needs CONFIG_VIRTIO_PMEM=y + CONFIG_FS_DAX=y	Documented; silent fallback to VirtIO-Block
No live migration	VM must be stopped to move it across hosts	Future work
No hot snapshots	VM must be stopped to snapshot the disk	Future work
No automated testing	Only manual testing	Unit + integration test suite
Linux host only	Requires Linux with KVM	By design
ebtables optional	L2 isolation only if ebtables is installed	Migration to nftables bridge in a future version
Compose IPs are literal	Changing cell topology requires regenerating compose	Placeholder syntax (\${PG_IP}) planned
Global nginx cache granularity	POST /admin/cache/purge affects all tenants; no per-tenant selectivity	ngx_cache_purge (third-party module) requires recompiling nginx
Community → enterprise upgrade post-creation	Requires manual PATCH /config of addons_path + restart; the enterprise share remount is not automatic	POST /edition/upgrade endpoint planned
Advanced headless bots bypass rate limit	Puppeteer-style bots with one request every 9s are not brute-force, fall outside the rate limit	CF Bot Fight Mode / fail2ban User-Agent pattern / IP reputation

Roadmap

Implemented in v1.1:

- `mt-compose-gen.sh` auto-generates `nkr-compose.yml` ✓
- VirtIO-FS for directory sharing with DAX ✓
- Prometheus exporter (`nkr serve`) ✓

- ebttables L2 isolation ✓
- Per-tenant `statement_timeout + conn_limit` ✓
- `cgroupv2 cpu.max + cpu.max.burst` bursting ✓

Implemented in v1.2:

- `vmlinux` ELF loader (~20 ms boot) ✓
- `io_uring` async I/O (~70% syscall reduction) ✓
- `VirtIO-PMEM + DAX` (~150–200 MB/VM page cache) ✓
- `Seccomp BPF jailer` ✓

Implemented in v1.3:

- Cell System (multi-stack with per-cell L2/L3) ✓
- `VirtIO-FS` with `DAX` replacing `VirtIO-9P` (3–5× faster) ✓
- `VirtIO-Balloon` (idle RAM reclamation) ✓
- `VirtIO-Console hvc0` (~2s coordinated shutdown) ✓
- `nkr cell clone` (atomic instance duplication with DB) ✓
- `nkr restart` (detached relaunch preserving original argv) ✓
- Zombie detection in `is_pid_alive()` (no more 90s in-vain waits) ✓
- Dynamic Nitro flow during compose boot ✓

Implemented in v1.4:

- `VirtIO-PMEM` active by default (`pmem: true`) ✓
- `skip_warmup: true` on clones (auto-injected by `append_compose_block`) ✓
- Filestore rename inside the guest (no host loop-mounts) ✓
- Netlink operation serialization (`flock /tmp/nkr-netlink.lock`) ✓
- `iptables -w 5` (waits for the kernel xtables lock) ✓
- API edge validation hardening ✓
- Internal cron janitor (every 5 min, cleans orphan mounts/cgroups/loops/locks) ✓

Implemented in v1.5:

- Privilege separation: root daemon (UDS) + unprivileged proxy (HTTP) ✓
- 16 documented REST endpoints ✓
- Systemd hardening of the proxy (empty `CapabilityBoundingSet`, `MemoryDenyWriteExecute`, ...) ✓
- Constant-time Bearer compare ✓
- Async init-db (202 + polling) ✓

- Better git error classification (401 auth, 404 not_found, 504 timeout, ...) ✓

Implemented in v1.6:

- Canonical template DB seed via `/web/database/create` with `lang=es_419` ✓
- Per-instance edition opt-in (community/enterprise filtering automation) ✓
- `admin_user_password` applied via JSON-RPC at tenant boot ✓
- Auto-explosion of multi-module OCA repos to `addons/<module>/` with `.nkr-source` tracker ✓
- Server-side nginx cache (`/web/static/*` 24h, `/web/assets/<hash>/*` 30d) ✓
- `POST /admin/cache/purge` endpoint ✓
- Edge nginx hardening (444 over `.php/.git/.zip` + CMS paths, rate limit on `/web/login`) ✓
- `sitecustomize.py` for real client IP in `werkzeug` + `gevent.pywsgi` log ✓
- `chown odoo:odoo /var/log/odoo` in `initramfs` before privilege drop ✓
- PgBouncer ram raised to 128 MB (was 64 MB, left little headroom) ✓

Implemented in v1.6.1:

- **Tier system** (`production / staging / dev`) with locked profiles for non-prod tiers ✓
— *note: the original `dev_mode=reload, qweb, xml` for DEV/STAGING was removed in v1.6.3 (see below); `dev_mode` is now empty in all tiers* ✓
- Canonical sizing (source of truth: `api.rs::derive_resources_for_tier`):
STAGING (1024 MB, workers=0, soft=600/hard=700, balloon boot/ACTIVE=256, IDLE=768), PROD ($\max(1024, 512 + 768 \cdot W)$ MB, soft=640 · W/hard=768 · W, balloon=0). DEV started at 768/400/512 and was raised to **1300 MB, soft=800/hard=1000** in v1.6.2 (see below) ✓
- HVC0 REL_OD channel — SIGUSR1 to the VM PID → vmm injects REL_OD\n over hvc0 → guest dispatch (SIGTERM+supervisor for threaded, SIGHUP master for prefork). ~3s without restarting the VM ✓
- Supervisor loop in `nkr-start.sh` (`while true; do exec odoo; sleep 1; done`) for threaded mode ✓
- `POST /reload` endpoint and default auto-reload after `POST /addons/git` ✓
- Cloudflare edge dual (proxied + DNS-only transparently coexisting, `set_real_ip_from` over CF ranges + `real_ip_header CF-Connecting-IP`) ✓
- Initramfs SIGTERM grace 25s → 5s (Odoo tenants drop from ~70s to ~25s on restart) ✓
- Async DELETE (closes the panel's HTTP timeout window) ✓
- Defensive guest DNS: bind-mount `/etc/resolv.conf` with 1.1.1.1 + 8.8.8.8, default route derived from `GUEST_IP` ✓

- `iptables -I FORWARD 1` so NKR rules sit above UFW ✓
- `KillMode=process` in the `systemd` unit (VMs survive `systemctl restart nkr`) ✓

Implemented in v1.6.2:

- **Dynamic IDLE/ACTIVE ballooning** per tier: the VM boots in IDLE state (max squeeze, `balloon=ram-256`), the panel marks ACTIVE via `POST /balloon` → `vmm` applies `set_target_mb(active) + IRQ config_change (~2s deflate)`. After 600s without renewal, automatic decay back to IDLE. PROD stays static (`balloon=0` always) to avoid deflate latency on traffic spikes ✓
- **SIGUSR2 safety check**: the daemon validates `/proc/<pid>/cmdline` before sending the signal (VMs launched with a pre-1.6.2 binary lack the handler and would be terminated — returns 202 `applied=false` instead) ✓
- **Double Hygiene** on `POST /addons/git`: recursive `git clean -ffdx` (parent + each submodule) + full wipe of `addons/` before re-populating. `addons/` becomes a deterministic mirror of the meta-repo. New `removed` field in the response (modules that were there before and are no longer in the current cycle) ✓
- **Strict 422 validation** of submodules: every `path = X` declared in `.gitmodules` must be either an Odoo module (manifest at root) or a grouper (with its own `.gitmodules`). Submodules without manifest nor grouper → 422 `submodule_no_manifest`. Doctrine: "the meta-repo is not a dumping ground for scripts" ✓
- **chattr +i cycle** on the master ext4: `nkr build` unlocks (`-i`) before writing and re-locks (`+i`) at the end. Covers any path under `/mnt/nkr/images/`. Reflinks (`cp --reflink=auto`) keep working against the immutable master ✓
- Canonical PROD RAM formula: $VM_RAM \geq 256 (OS) + 256 (master) + 768 \times workers$, with API-side validation (`400 ram_insufficient_for_workers`) ✓
- Faucet Rule: `workers > 4` → `balloon=0` forced in compose ✓
- `Workers=0` (threaded) **mandatory** in DEV/STAGING (no override allowed); `workers` configurable 1..16 only in PROD ✓

Implemented in v1.6.3:

- **dev_mode emptied in all tiers** — `reload` exhausts the guest's `fs.inotify.max_user_watches` (default 8192) recursing over `/usr/lib/python3/.../odoo/addons` → `OSError [Errno 28]` → Odoo dies `rc=1` → supervisor respawn loop → `:8069` never comes up (postmortem in `BUG_inotify_dev_mode.md`). `qweb,xml` activates Odoo's internal watchdog that recompiles QWeb/XML templates on every request (including nginx keepalives every 30s) → CPU/GC pressure correlating with host-side `nkr` hangs. The canonical hot-reload is `REL_OD` over `HVC0`; `dev_mode=` is now empty for production *and* dev/staging ✓
- **DEV profile bumped to 1300 MB** (soft/hard 800/1000) after `Server memory limit`

reached cycling with Odoo 19 + ~31 custom modules in threaded mode (the 400 MB soft was unreachable under normal DEV load) ✓

- **Watchdog** (`watchdog.rs`) — daemon thread, TCP probe of :8069 per running tenant every 15s; after `HUNG_THRESHOLD_SECS=60` consecutive misses, auto-restart via `api::handle_action`. Bypass via `NKR_WATCHDOG_DISABLED=1`. **Currently shipped disabled** by request (auto-restarts interfered with the panel pushing changes actively) ✓
- Faster restart: per-module diff in `POST /addons/git` + reduced timer drain ✓

Implemented in v1.6.4 (security/operability sprint):

- **HMAC-signed SSO** — `POST /cells/{cell}/instances/{name}/sso {user}` mints `https://<dns>/nkr-sso?u=<login>&exp=<ts>&sig=<hmac_sha256>` (30s TTL), signed with a per-tenant 256-bit key written by `cell.rs::rewrite_odoo_conf_full` into `odoo.conf [nkr_sso] secret` (not `[options]` — avoids Odoo's unknown option warning; legacy `nkr_sso_secret` in `[options]` still honored as fallback). The Odoo module `nkr_sso` verifies the HMAC constant-time + checks expiry + creates a sudo session (`request.session.session_token = user._compute_session_token(sid)`) — **the user's password never leaves the host**. Rotate = edit `[nkr_sso] secret` + `POST /actions {restart}` ✓
- **systemouts-addons** — `cells/<cell>/systemouts-addons/` mounted RO in every instance as `/mnt/systemouts-addons`, inserted in `addons_path` *before* `/mnt/extra-addons` (the tenant's own addons/) → a client module with the same name can't shadow an internal one. `POST /addons/git` never touches it → the client can't see or overwrite it. One copy per cell, RO. Holds `nkr_sso/` today; pre-installed in each `<cell>-odoo-template` (code + state=installed in the template DB) → clones inherit both via `CREATE DATABASE ... TEMPLATE` + the cell-level RO share. The secret is regenerated fresh per tenant ✓
- **Async POST /instances** — validates synchronously (all 4xx immediately), dispatches the clone in a background thread, returns `202 {nkr_name, poll}`. The panel polls `GET /cells/{cell}/instances/{name}/create-status` until `status=ready|failed`; status file at `/mnt/nkr/cells/{cell}/.nkr-creates/{name}.json` (cell-level, survives the clone's rollback). Eliminates the false 504 when PROD prefork boots ~140s — past the panel's/Cloudflare's HTTP timeout ✓
- **POST /diag** — captures host-side stacks/wchan/cpu of the tenant's nkr process threads (text/plain, ~50ms, idempotent) — pre-restart forensics ✓
- **Initramfs /mnt tmpfs** — after mounting the RO master rootfs, the initramfs mounts a tmpfs over `/newroot/mnt` → any new virtio-fs share with a fresh guest path under `/mnt/*` (`mount -t virtiofs ... /mnt/foo`) does `mkdir -p` over the tmpfs and "just works" **without rebuilding the master rootfs** (which is RO in the guest) ✓
- **POST /reload fix for workers=0** — the HVC0 watcher reads `workers = N` from

the guest's `odoo.conf`: `empty/0` → `pnkr -TERM /usr/bin/odoo` (the `pnkr -start.sh` supervisor loop respawns it with fresh code); `>0` → `pnkr -HUP master` (prefork). No VM downtime in either case. Obsoletes the workaround "use `POST /actions {restart}` for `workers=0`" ✓

- **Dynamic-balloon fix for `tier=dev`** — `vmm.rs::configure_linux_boot` now advertises the balloon `virtio_mmio.device` to the guest when dynamic ballooning is configured (`balloon_idle_mb ≠ balloon_mb`), not only when `balloon_mb > 0`. Before, `DEV` (which boots `ACTIVE` with `balloon_mb=0`) never got the device on the `cmdline` → the guest never attached the driver → the `IDLE`-decay `set_target_mb(256)` was a no-op. Now it actually inflates. `state::update_balloon_mb` writes the runtime target to the VM state file on each `ACTIVE ↔ IDLE` transition → `pnkr_balloon_mb / pnkr_ps` reflect the current target, not the boot value ✓
- **Per-instance boot console log** — each VM writes guest serial console (initramfs echos + `dmesg`) + VMM `stderr` to `<instance>/.<config>-vm-boot.log` (truncated each boot) — diagnoses `virtio-fs` mounts, guest panics, `cmdline` truncation ✓
- **Kernel `cmdline` truncation fix** — `COMMAND_LINE_SIZE` is small (~1024 B); with many `virtio-fs` shares the `cmdline` truncated (lost `init= / pnkr.ip=` at the end). Fix: omit the redundant `rootfs pnkr.fs0/fsm0/fsr0` params (the initramfs mounts the `rootfs` via `pnkr.rootfs=`, not `pnkr.fs0=`) and emit `pnkr.fsr{i}=` only when `ro` (absence ⇒ `rw`). ~60 B of headroom ✓

Implemented in v1.6.5–v1.6.9 (production operation on real hardware):

- **Deterministic code reload (`REL_OD`, v1.6.9)** — the guest supervisor writes Odoo's PID to `/tmp/odoo.pid` (`su -c 'echo $$ > /tmp/odoo.pid; exec python3 odoo'`); the `HVC0` watcher (outside the `chroot`) issues a direct `kill -KILL $pid`, with no `pnkr -f` and no `SIGTERM` grace. Consistent ~7s commit → reload cycle under load (websocket + cron). Live diagnosis in `<instance>/logs/pnkr-watcher.log` ✓
- **`compose up -d detach` fix (v1.6.9)** — spawned VMs `setsid()` and write `stdout/stderr` straight to the boot log (`Stdio::from(File)`, no pipes) → the `compose` process exits cleanly and VMs end with `PPid=1`. Eliminates the bug where `compose up -d` hung in `child.wait()` holding pipes and a `pnkr -f 'pnkr compose'` killed all VMs in cascade ✓
- **Watchdog `ENABLED` + `REL_OD`-aware (v1.6.8)** — threshold 60s → 120s (false positives on legitimate deploys); `note_reload()` raises the effective threshold to 180s during a 240s grace window after a reload; **pre-restart forensics** (`capture_hang_forensics`) dumping to `/mnt/pnkr/hangs/<name>-<ts>.txt` the per-thread VMM stacks/`wchan`, `cgroup` memory. `{current, max, events}` + `PSI {memory, cpu, io}.pressure`, `--ram` vs `memory.max`, and the guest's internal RAM. Shipped `ENABLED` by default (validated by recovering a real hang) ✓
- **Per-tier PostgreSQL connection policy (v1.6.9)** — `dev/staging` → direct PG

(db_maxconn=10), prod → PgBouncer (db_maxconn=15); per-cell PgBouncer max_client_conn=250 / default_pool_size=20 / reserve_pool_size=5. After root-causing the chronic dev/staging hang as Odoo connection-pool exhaustion (not D-state/memory/virtio-fs); validated with 0 PoolError + ~28 ms ✓

- **Initramfs -72% (monolithic kernel)** — removed the embedded nkr-watcher binary (400 KB) + the lib/modules/*.ko (3 MB) from the generated initramfs; the custom kernel is monolithic with all drivers built in. 4.20 MB → 1.17 MB per VM (~63 MB freed with 21 VMs) ✓
- **Symlinking the pg/pgbouncer rootfs to the immutable master** — they share page cache via DAX across cells; ~400 MB RSS + 914 MB disk saved ✓
- **Measured PSS baseline (Phase 0)** — idle dev 223 MB / staging 197 MB / prod workers=2 343 MB / dev under traffic 524 MB (KSM=0); the DAX rootfs alone contributes ~31% of the sharing ✓
- **Snapshot/restore evaluated and dropped** — ~25–75 MB/idle VM, ~0 under traffic: not worth the 3–4 week refactor; documented as an early-telemetry study ✓
- **Splitting client addons across 2 directories** — the daemon mounts two independent client addon directories (/mnt/extra-addons and /mnt/extra-addons-2) plus the internal RO systemouts-addons, to support two client repos per tenant ✓
- **Filestore rename fix on clones** — idempotency by directory existence + self-heal, removing the fragile marker that traveled inside the cloned ext4 and broke clone-of-clone ✓

Remote Development Workspaces (Dev-Cells) — under construction, core delivered:

- dev-workspace tenant class with an independent WorkspaceResources (Odoo's cell::Tier enum left untouched) ✓
- Spartan master rootfs (Nkrfile.dev-workspace) with Python/Node/git/build-essential/postgresql-client/code + preinstalled PostgreSQL; nkr build → ~1.2 GB physical ✓
- DB hydration via reflink of the cell-level golden PGDATA (full isolation, O(1)) ✓
- Per-dev persistent profile via RW virtio-fs share (/opt/nkr/dev-profiles/<user>/), fixed UID/GID 1000 ✓
- Bootstrap pipeline with JSON state machine + fail-fast + guillotine on phase=failed ✓
- Guest→host channel via virtio-fs + inotify (stateless host, drop-on-failure) ✓
- Real VM spawn: TAP + nkr-devbr0 bridge + isolated 10.99.0.0/24 subnet with iptables DROP ✓
- API: POST /workspaces, GET /status, POST /rebuild-db, DELETE, POST /_gc (with editor_target web/desktop) ✓
- Security: host → panel webhook with existing token + IP whitelisting; per-dev Anthropic key

(no master key) ✓

- *Pending*: initial golden PGDATA, inactivity GC (12h) + daily 04:00 destruction, observability

High priority:

- End-to-end validation with 5 cells × 20 Odoos on a single host
- Automated tests over the HTTP API (today only unit tests in `fsutil`)
- Compose IP placeholders (`${PG_IP}`, `${PGB_IP}`)
- Migration to nftables bridge (replace ebtables)

Medium priority:

- `POST /edition/upgrade` endpoint (community → enterprise without recreate)
- Per-tenant granularity in cache purge (via recompiled `ngx_cache_purge`)
- Multi-vCPU support
- Automated PostgreSQL backup per tenant
- Build-time QWeb pre-compile in the template (eliminate the 5s of first boot)
- **Ops-only global/host view** — host-level metrics (`/proc/meminfo`, `/proc/stat`, `statvfs` → "how much RAM/CPU/disk is left on the box") + a dashboard joining that with the per-VM aggregate. (All *tenant* metrics — host-side per-VM, `nkr_up{vm, cell, tier}`, build info, cluster totals, plus the per-instance JSON endpoint with cgroup CPU/mem, disk via `du`, and guest-internal RAM via the virtio-balloon stats `virtqueue` — shipped in v1.6.4.)
- v17 cell parity: pre-install `nkr_sso` into the `odoo-v17` template (the `/mnt/systemouts-addons` mountpoint is already in the v17 master rootfs)

Low priority:

- Live migration between servers
- Hot snapshots without stopping the VM
- KSM with hybrid memory layout (anon-private + memfd-shared) — only if 110+ VMs don't fit in 32 GB

Conclusion

NKR demonstrates that it is possible to achieve **container-level density and operational simplicity** with **VM-level isolation and resource guarantees**, in roughly 21,700 lines of Rust, compiling to two binaries (~2.4 MB the daemon, ~660 KB the proxy) with no runtime dependencies.

Version 1.3 raised the density ceiling to 103+ Odoo instances on a 32 GB server via VirtIO-FS +

DAX, VirtIO-Balloon, VirtIO-Console hvc0, the Cell System, and `nr cell clone`. Versions 1.4–1.6 transform NKR from a manually operated *runtime* into an **API-driven SaaS platform**: the root daemon exclusively exposes an internal UDS and the entire HTTP frontend lives in an *unprivileged* process with no capabilities, with REST endpoints covered by double validation (proxy + daemon), per-IP rate limiting, server-side nginx static cache, scanner *hardening* (444 over CMS/legacy paths), and a tenant provisioning pipeline that handles DB, DNS, certificates, admin user password, and OCA addons in a 4-call sequence. Versions 1.6.1–1.6.4 close the operational doctrine: the **tier system** decouples iteration profiles (DEV/STAGING with threaded `workers=0`, supervisor loop, **empty dev_mode** — the `REL_OD/HVC0` reload replaces `dev_mode=reload`, which is incompatible with `virtio-fs` and exhausts guest `inotify`) from production operation (PROD with `prefork` multi-worker, no auto-reload); the **HVC0 REL_OD channel** resolves the `virtio-fs+inotify` limitation by reloading workers in ~3s without restarting the VM (threaded → `kill -TERM` + supervisor respawn; `prefork` → `kill -HUP` master); **dynamic IDLE/ACTIVE ballooning** with automatic decay maximizes density under realistic load (32 GB host → ~110 idle DEV instances, deflated on demand); **Double Hygiene + strict 422** turn addons/ into a deterministic mirror of the meta-repo, eliminating drift between `git push` and the tenant filesystem; the **chattr +i** cycle on the master ext4 closes an entire class of failures from accidental mutation of the shared rootfs; **HMAC-signed SSO** + the **systemouts-addons** cell-level read-only addons path let the panel auto-login users into any tenant (30s-TTL signed URL, password never leaves the host) using an internal Odoo module the client cannot see or override; **async POST /instances + create-status** polling remove the false 504 on slow PROD boots; and an **initramfs /mnt tmpfs** makes new `virtio-fs` shares "just work" without rebuilding the immutable master. Versions 1.6.5–1.6.9 mature the operation on real hardware: the **REL_OD code reload** becomes deterministic (`/tmp/odoo.pid + direct kill -KILL`, a consistent 7s) and the ~~`compose up -d detach`~~ can no longer kill all VMs in cascade; the ~~`watchdog`~~ is enabled, `REL_OD`-aware, and dumps forensics that — together with a deterministic repro — let us root-cause the chronic dev/staging hang as **Odoo connection pool exhaustion** (not kernel or memory), resolved with a ~~`per-tier PostgreSQL connection policy`~~; and a ~~`telemetry Phase 0`~~ measured a real PSS baseline (200–340 MB/VM, with the DAX rootfs contributing 31% of the sharing), justified the **72% initramfs reduction** (monolithic kernel) and the ~~`symlinking of the infrastructure rootfs to the master`~~ (400 MB RSS + 914 MB disk), and led to **dropping** the snapshot/restore refactor as not worth its cost.

For operators managing dozens of SaaS tenants on a single server, NKR delivers a fundamentally different balance from Docker, traditional VMs, or a hosted PaaS:

- **Each tenant gets hardware isolation**, not just namespace separation
- **Each tenant gets guaranteed resources**, not shared CPU and memory pools — the panel only picks `workers=N` and NKR derives ram, chrs, soft/hard memory limits from a deterministic formula
- **The operator keeps Docker workflows**, with familiar build, run, and compose patterns
- **Infrastructure consolidates**: 1 PostgreSQL + 1 PgBouncer per cell + N Odoos + 1 nginx on the host; instead of N fully-overlapping stacks
- **The external panel manages everything via REST**: create tenant, clone OCA repos with

auto-explosion, apply admin password, provision DNS+TLS, change workers, run sandboxed SQL, cursor-tail logs — without SSH-ing to the host, behind an **IP whitelist + a per-panel Bearer token**

- **The same machinery serves development, not only production: Remote Workspaces (Dev-Cells)** reuse reflink, virtio-fs, the initramfs, and ballooning to give each developer an ephemeral micro-VM with a fresh DB (golden PGDATA reflink), a persistent profile, and a VS Code editor over an outbound tunnel — without opening an inbound port, on an isolated `10.99.0.0/24` network with daily 04:00 destruction

NKR is software with a purpose. Rather than trying to be a general-purpose hypervisor like QEMU or a general-purpose container platform like Kubernetes, NKR focuses on a specific, high-value workload pattern: **dense multi-tenant Odoo SaaS on bare metal**. That focus makes it small enough to be fully understood, small enough to be audited line by line, fast enough to boot in seconds, and robust enough to serve 100+ production tenants behind a single control panel.

\newpage

Appendix A: Technology Stack

Component	Technology	Version	Since
Language	Rust	Edition 2021	v1.0
KVM interface	kvm-ioctls	0.19	v1.0
KVM bindings	kvm-bindings	0.10	v1.0
Guest memory	vm-memory (GuestMemoryMmap)	0.14	v1.0
Kernel loader	linux-loader (bzImage + ELF)	0.11	v1.0 / v1.2
VirtIO queues	virtio-queue	0.12	v1.0
CLI	clap (derive)	4.x	v1.0
Serialization	serde + serde_yaml + serde_json	1.x / 0.9 / 1.x	v1.0
System utilities	vmm-sys-util	0.12	v1.0
Async I/O	io-uring	0.6	v1.2
Guest kernel	Linux vmlinux ELF / bzImage	6.6.117-0-virt	v1.0

Appendix B: Source Code Metrics

(Approximate, as of v1.6.9 — ~21, 700 lines of Rust total across src/.)

Module	File	Lines	Responsibility
API handlers	api.rs	~3,930	IPC dispatch, instance lifecycle (sync validation + async clone), DNS, init-db, modules, psql sandbox, cache purge, handle_sso/handle_diag/handle_create_status, derive_resources_for_tier
VMM engine	vmm.rs	~2,860	KVM init, PIT2, ELF/bzImage loader, MMIO, cgroups, ebtables, PMEM slot, seccomp, hvc0 shutdown + REL_OD, dynamic balloon state machine, cmdline assembly
Cell system	cell.rs	~2,740	Cell registry, bridge management, instance directories, clone_instance, edition opt-in, rewrite_odoo_conf_full (HMAC secret, systemouts-addons share + addons_path, dev_mode empty)
HTTP proxy	bin/nkr_api_server.rs	~2,600	HTTP → IPC translation, validation, body limits, thread pool, auth, /metrics, pylibs PUT
Compose	compose.rs	~1,670	YAML, orchestration, health checks, daemon mode, Nitro/warmup flow, per-instance boot-console log
Initramfs	initramfs.rs	~1,080	Boot environments (per-instance template), /mnt tmpfs, FS/PMEM/virtiofs module loading, hvc0 watcher (REL_OD

			mode detection), sitecustomize.py injection
VirtIO-FS	virtio_fs.rs	~685	VirtIO-FS (DAX, vhost-user) replacing 9P
Metrics	metrics.rs	~630	/proc telemetry, KSM, Prometheus exporter
Main	main.rs	~510	Entry point, full dispatch including Cell/Clone
State	state.rs	~450	VM registry, lifecycle tracking, zombie detection, nkr ps, update_balloon_ mb
Balloon	balloon.rs	~400	VirtIO-Balloon, MADV_DONTNEED idle page eviction
Block	block.rs	~390	VirtIO-block, io_uring async I/O + sync fallback
Network	net.rs	~365	VirtIO-net, TAP backend, RX/TX threads, io_uring TX
CLI	cli.rs	~360	Full CLI: run/ps/stop/restart/nitro /compose/pull/build/sta ts/ksm/serve/cell
API HTTP helpers	api_http.rs	~350	Validation regexes, route helpers
Janitor	janitor.rs	~350	Internal cron, orphan cleanup
Fsutil	fsutil.rs	~300	ext4 creation with chattr +C, integrity checks
IPC	ipc.rs	~285	Length-prefixed JSON wire over UDS (incl. Sso/Diag/GetCreat eStatus)
PMEM	pmem.rs	~280	VirtIO-PMEM + DAX, mmap(MAP_SHARED) , FLUSH handler
IPC server	ipc_server.rs	~250	UDS request loop, dispatch to api : : *

Console	console.rs	~165	VirtIO-Console (hvc0) device
Watchdog	watchdog.rs	~210	TCP : 8069 probe per tenant, auto-restart (120s threshold, REL_OD grace 240s/180s), pre-restart forensics to /mnt/nkr/hangs/ (enabled by default)
Seccomp	seccomp.rs	~135	BPF jailer (~120-syscall allowlist)
Total		~21,700	(~19,100 in the daemon nkr binary + ~2,600 in the unprivileged nkr - api-server)

Appendix C: Quick Start

```
# Build NKR from source
cargo build --release
# Binary: target/release/nkr (~2-4 MB)

# — Pull and Build —
# Download PostgreSQL image and create disk
sudo ./target/release/nkr pull postgres:15 postgres.ext4 --size-mb 2048

# Build from Nkrfile
sudo ./target/release/nkr build -f Nkrfile.odoo -o odoo.ext4 --size-mb 4096

# — Basic execution —
# Run a micro-VM
sudo ./target/release/nkr run \
  --disk postgres.ext4 --ram 512 --chrs 1 --id 1 --port 5432:5432

# Run with live VirtIO-FS sharing
sudo ./target/release/nkr run \
  --disk odoo.ext4 --ram 256 --chrs 1 --id 2 \
  --share /opt/modules:/mnt/extra-addons \
  --share
/mnt/nkr/cells/empresa_prueba/instances/empresa_prueba-odoo-01/config:/etc/odoo

# Run with VirtIO-PMEM + DAX (~150-200 MB RAM saved)
sudo ./target/release/nkr run \
  --disk odoo.ext4 --ram 256 --chrs 1 --id 3 --pmem

# Run with VirtIO-Balloon (reclaim 200 MB from idle VM)
sudo ./target/release/nkr run \
  --disk odoo.ext4 --ram 512 --chrs 1 --id 4 --balloon-mb 200

# — Lifecycle —
sudo ./target/release/nkr ps          # list active VMs
sudo ./target/release/nkr stats      # live CPU/RAM/IO/NET
sudo ./target/release/nkr stop empresa_prueba-odoo-01  # clean shutdown
via hvc0
```

```

sudo ./target/release/nkr restart empresa_prueba-odoo-01      # stop + relaunch
detached

# — Nitro (temporary CPU unblock) —————
sudo ./target/release/nkr nitro empresa_prueba-odoo-01 --duration 10m

# — KSM (page deduplication) —————
sudo ./target/release/nkr ksm on
sudo ./target/release/nkr ksm status

# — Prometheus metrics —————
sudo ./target/release/nkr serve --port 9090
curl http://localhost:9090/metrics

# — Cell System —————
# Create a cell (registers cell_id, creates nkr-br1 bridge, directories)
sudo ./target/release/nkr cell create empresa_prueba --odoo-version 17.0

# List all cells
sudo ./target/release/nkr cell ls

# Bring up the full stack (requires nkr-compose.yml in the cell dir)
sudo ./target/release/nkr cell up empresa_prueba -d

# View active VMs in a cell
sudo ./target/release/nkr cell ps empresa_prueba

# Stop all VMs in a cell
sudo ./target/release/nkr cell down empresa_prueba

# Remove cell from registry (data preserved)
sudo ./target/release/nkr cell destroy empresa_prueba

# — Instance Cloning —————
# Full clone: files + DB + compose block
sudo ./target/release/nkr cell clone empresa_prueba-odoo-01 empresa_prueba-
odoo-04

# Safe smoke test: files only, no DB or compose modification
sudo ./target/release/nkr cell clone empresa_prueba-odoo-01 empresa_prueba-
odoo-04 \
    --no-db --no-compose

# — Compose —————
sudo ./target/release/nkr compose up -f nkr-compose.yml -d
sudo ./target/release/nkr compose down -f nkr-compose.yml
sudo ./target/release/nkr compose ps

# — HTTP API (modern operation, v1.6) —————
# The daemon runs as a systemd service; the external panel manages everything
via REST.

# Start root daemon + unprivileged proxy:
sudo systemctl enable --now nkr.service                # root daemon, UDS only
sudo systemctl enable --now nkr-api-server.service      # HTTP proxy localhost:9090

# Health (no auth):
curl http://nkr-host:9090/api/v1/health
# → {"ok":true,"version":"1.6.9"}

# Create production tenant (with admin password applied at boot):
curl -X POST http://nkr-host:9090/api/v1/instances \
    -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
    -d '{

```

```

    "nkr_name": "client-42",
    "mode": "production",
    "odoo_version": "19.0",
    "edition": "community",
    "workers": 2,
    "admin_passwd": "MasterPwd-xxxxxxxxxxxxxx",
    "admin_user_password": "AdminLoginPassword-yyyy",
    "dns": "client-42.example.com"
  }'

# Provision DNS + Let's Encrypt cert + nginx vhost:
curl -X POST http://nkr-host:9090/api/v1/cells/odoo-v19/instances/client-42/dns \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"dns":"client-42.example.com","enable_websocket":true}'

# Clone multi-module OCA repo (auto-explodes to addons/<module>):
curl -X POST
http://nkr-host:9090/api/v1/cells/odoo-v19/instances/client-42/addons/git \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"repo_url":"https://github.com/OCA/account-financial-
tools.git","ref":"19.0"}'
# → { "module_count": 23, "modules": [...] }

# Restart so Odoo reloads manifests:
curl -X POST
http://nkr-host:9090/api/v1/cells/odoo-v19/instances/client-42/actions \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"action":"restart"}'

# Tail logs (cursor-paginated, supports long-poll):
curl -H "Authorization: Bearer $TOKEN" \
  "http://nkr-host:9090/api/v1/cells/odoo-v19/instances/client-42/logs?tail=100"

# Wipe nginx cache (after updating logo or assets in /web/static/):
curl -X POST -H "Authorization: Bearer $TOKEN" \
  http://nkr-host:9090/api/v1/admin/cache/purge

```

NKR is proprietary and confidential software. All rights reserved. No reproduction, distribution, or disclosure without the owner's written permission.

© 2026 Antonio Vargas. All rights reserved.